

Autonomous Navigation System Using Sensor Fusion

Department Lippstadt 2

Project Work

for obtainment of the degree of
Bachelor of Engineering

submitted by

Md Rafiul Islam

Electronic Engineering

Mat.Nr.: 1232714

`md.rafiul-islam@stud.hshl.de`

June 01, 2026

First supervisor: Prof. Dr. Achim Rettberg

Second supervisor: M. Sc. Ali Alhalabi

Abstract

Reliable self-localisation is a key requirement for autonomous mobile robots, particularly in indoor environments where satellite-based positioning cannot provide dependable localisation. This project investigates an encoder-free navigation approach for a low-cost ground vehicle by combining inertial measurements with LiDAR-based distance information. The aim is to estimate the vehicle motion using onboard sensing and EKF-based sensor fusion rather than relying on wheel-encoder odometry.

The experimental platform is built around a Raspberry Pi, a GY-87 inertial measurement unit, a two-dimensional LiDAR sensor, and two DC gear motors driven through an L298N H-bridge. The Raspberry Pi performs the control and sensor processing tasks, while the vehicle motion is estimated using an Extended Kalman Filter. The filter combines heading information from the inertial sensors with LiDAR wall-distance correction to reduce the drift that normally appears in dead-reckoning-based mobile robot navigation. The implemented system was tested using an L-shaped, half-rectangular motion sequence consisting of a first forward motion, a right turn, and a second forward motion. This trajectory represents the main motion elements required for a full rectangular path, since a complete rectangle can be formed by repeating the same straight-line and turning behaviour. During the experiment, telemetry data including position, heading, travelled distance, velocity, LiDAR distance, and motion phase were transmitted from the Raspberry Pi to a host computer using UDP communication. For analysis and validation, the received data were logged and post-processed in MATLAB. In addition, the same UDP telemetry stream was integrated into a ROS 2 and RViz visualisation pipeline on Ubuntu, allowing the robot state, path, and motion behaviour to be monitored during operation. A separate simulation comparison between motion estimation with and without EKF correction was also used to evaluate the benefit of the sensor fusion approach.

The results show that the proposed LiDAR-IMU sensor-fusion system can support encoder-free autonomous navigation for structured indoor motion. Although the practical experiment was carried out on a half-rectangular trajectory, the implemented control, estimation, communication, and visualisation pipeline is directly extendable to a complete rectangular motion test by continuing the same sequence of forward motions and right-angle turns.

Contents

Abstract	1
1 Introduction	4
1.1 Motivation	4
1.2 Aim of the Project	4
1.3 Scope of the Work	5
1.4 Structure of the Report	6
2 Fundamentals	7
2.1 Mobile Robot Localisation	7
2.2 LiDAR Distance Measurement	8
2.3 Inertial Measurement Unit	8
2.4 Extended Kalman Filter	10
2.5 Embedded Controller and Motor Drive	11
2.6 UDP Communication, MATLAB Analysis, and RViz Visualisation . . .	12
2.7 Power Supply and Practical Limitations	13
3 Related Work	14
4 Analysis	18
4.1 System Requirements	18
4.2 Latest Raspberry Pi to MATLAB Data	19
4.3 Ubuntu, ROS2/RViz, and MATLAB Visualisation Data	20
4.4 Technical Constraints	27
4.5 Evaluation Criteria	28
4.6 Data and Visualisation Workflow	29
5 Design and Implementation	30
5.1 System Architecture	30
5.2 Mechanical Platform Design	31
5.3 Electrical System Design	32
5.4 Software and Communication Architecture	34

5.5	Sensor Fusion and Motion Sequence	36
5.6	Motor Control Implementation	37
5.7	Integration and Testing Strategy	38
5.8	Summary	39
6	Evaluation	40
6.1	Test Environment, Data Sources, and Methodology	40
6.2	Latest Real Motion Result	41
6.3	Distance, Velocity, and Yaw Behaviour	42
6.4	LiDAR, EKF, and Motor-Control Behaviour	44
6.5	Visualisation Pipeline and EKF Comparison	47
6.6	Limitations and Overall Assessment	53
7	Summary and Outlook	55
7.1	Summary	55
7.2	Outlook	56
8	Conclusion	58
A	Appendix	62
A.1	Latest Experiment Data	62
A.2	Project Files	62
A.3	Report Figures	63
A.4	Future Full-Rectangle Dataset	63

Chapter 1

Introduction

1.1 Motivation

Autonomous mobile robots require reliable self-localisation in order to move safely and repeatably in indoor environments. For low-cost robotic platforms, this task is difficult because accurate wheel encoders, external positioning systems, or GPS-based localisation are often not available. In such cases, the robot must estimate its own motion from onboard sensors while dealing with motor slip, sensor noise, heading drift, and limited computational resources.

This project focuses on an encoder-free autonomous ground vehicle that uses a Raspberry Pi as the main processing unit. The vehicle combines a GY-87 inertial measurement unit with LiDAR-based distance information in order to estimate its motion during a structured indoor navigation task. The main motivation is to investigate whether a low-cost platform can perform useful autonomous motion by combining inertial sensing, ranging measurements, motor control, data logging, and live visualisation..

A single sensor is not sufficient for this task. Inertial sensors provide fast motion and heading information, but their estimates drift over time. LiDAR measurements provide useful distance information to nearby walls or obstacles, but they depend on the surrounding environment and can be affected by noise or missing reflections. By combining both sensor types, the system can reduce the individual limitations of each sensor and obtain a more stable estimate of the robot state. This principle is implemented in this work using an Extended Kalman Filter for sensor fusion.

1.2 Aim of the Project

The aim of this project is to design, implement, and evaluate a low-cost autonomous vehicle that can estimate and visualise its motion without using wheel encoders. The robot is required to execute a structured motion sequence consisting of straight-line

movement and a right-angle turn. In the practical experiment, this was implemented as an L-shaped, half-rectangular trajectory with the phases `forward_1`, `turn_right`, and `forward_2`. This trajectory contains the essential motion elements of a complete rectangular path, because a full rectangle can be generated by repeating the same straight and turning behaviour.

The project also aims to create a complete experimental pipeline around the robot. The Raspberry Pi collects sensor data, estimates the vehicle state, and transmits telemetry data through UDP communication. The received data are then used for MATLAB-based plotting and analysis. In addition, the same telemetry stream is integrated into a ROS 2 and RViz pipeline on Ubuntu, so that the robot path and state can be monitored visually during operation. Therefore, the work does not only focus on the robot movement itself, but also on the full process of measurement, communication, logging, post-processing, and visual validation.

A further aim is to evaluate the benefit of the EKF-based correction compared with motion estimation without such correction. For this purpose, simulation results are used to compare the estimated trajectory with and without EKF support. The real experiment demonstrates the implemented half-rectangular motion, while the simulation comparison helps to show why sensor fusion is important for reducing drift and improving position estimation.

1.3 Scope of the Work

The scope of this work includes the implementation of the Raspberry Pi robot, the integration of the GY-87 inertial sensor and LiDAR distance measurement, the motor control using an L298N H-bridge, and the development of the EKF-based state estimation approach. The system also includes UDP-based data transmission from the robot to a host computer, MATLAB-based evaluation of the recorded data, and ROS 2/RViz-based visualisation of the robot motion.

The practical validation in this report is limited to the available half-rectangular test. This means that the robot was not evaluated through a complete four-leg rectangular trajectory in the latest real experiment. However, the tested L-shaped motion already contains the two basic movement primitives required for a full rectangular path: forward motion and a right-angle turn. For this reason, the implemented method is applicable to a complete rectangular test, but the full rectangular experiment itself is treated as a future extension rather than as a completed measurement result.

1.4 Structure of the Report

Chapter 2 introduces the technical background required for this work, including mobile robot localisation, LiDAR sensing, inertial measurement, motor control, and Extended Kalman Filtering. Related work is discussed in Chapter 3, where existing approaches in mobile robot localisation and sensor fusion are summarised in one coherent chapter. Chapter 4 describes the system requirements and the practical constraints that influenced the design of the robot and the experimental setup.

The design and implementation of the system are presented in Chapter 5. This includes the hardware structure, the software workflow, the EKF-based estimation approach, UDP communication, MATLAB logging, and ROS 2/RViz visualisation. Chapter 6 evaluates the implemented system using the latest half-rectangular experimental data and the EKF comparison results. Finally, Chapter 7 summarises the project and presents possible future extensions, including the execution of the complete rectangular trajectory. The conclusion is presented as a separate final chapter.

Chapter 2

Fundamentals

This chapter introduces the technical background required to understand the implemented autonomous navigation system. The project is based on a low-cost Raspberry Pi robot that estimates its motion without wheel encoders by combining inertial sensing and LiDAR-based distance correction. The latest practical experiment is an L-shaped, half-rectangular motion sequence consisting of a first forward motion, one right-angle turn, and a second forward motion. This chapter therefore focuses only on the fundamentals that are directly relevant to this latest implementation: mobile robot localisation, LiDAR sensing, inertial measurement, Extended Kalman Filtering, motor control, UDP communication, MATLAB analysis, and ROS 2/RViz visualisation.

2.1 Mobile Robot Localisation

Mobile robot localisation is the process of estimating the position and orientation of a robot relative to its environment. In many robotic platforms, wheel encoders are used to measure wheel rotation and estimate travelled distance. In this project, however, the robot is designed as an encoder-free platform. This means that the position, velocity, and heading of the vehicle must be estimated from onboard sensors instead of direct wheel rotation measurements.

The main difficulty of encoder-free localisation is drift. If the robot motion is estimated only from acceleration or angular-rate measurements, small sensor biases accumulate over time and cause the estimated state to move away from the true motion. This problem is especially important in indoor environments, where GPS is not available. For this reason, the system combines inertial measurement with LiDAR-based distance correction. The latest real experiment uses an L-shaped trajectory, but the same movement primitives are also required for a complete rectangular path: straight-line movement and right-angle turning.

2.2 LiDAR Distance Measurement

LiDAR, short for Light Detection and Ranging, is an active sensing method that measures distance by emitting light and evaluating the reflected signal [13]. In a time-of-flight LiDAR system, the distance can be described by

$$d = \frac{c \cdot t}{2}, \quad (2.1)$$

where d is the measured distance, c is the speed of light, and t is the round-trip travel time of the emitted pulse. The division by two is necessary because the light travels from the sensor to the object and back to the sensor.

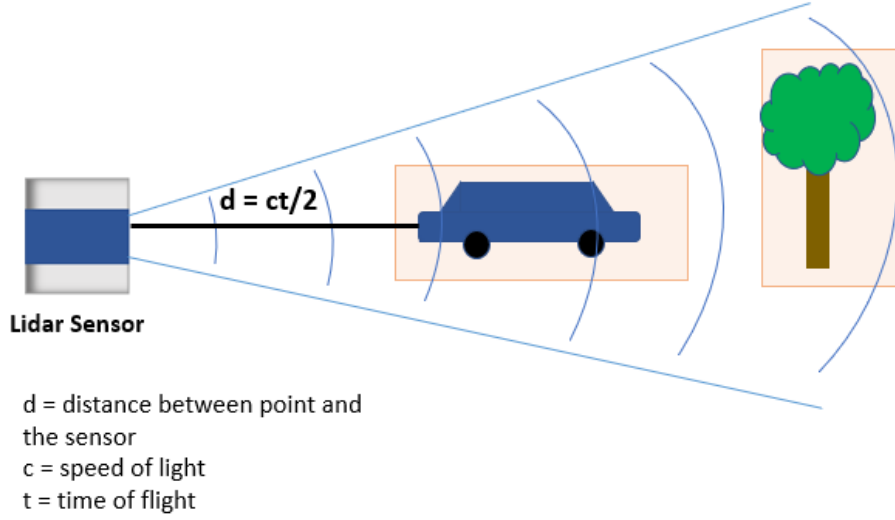


Figure 2.1: Basic LiDAR time-of-flight principle. The distance is estimated from the travel time of the emitted and reflected light pulse.

In this project, the LiDAR is used as a local ranging sensor rather than as a complete mapping sensor. It provides distance information to nearby walls or objects and is especially useful for correcting position drift during straight motion. Compared with pure inertial integration, LiDAR distance information does not accumulate drift in the same way. However, it can be affected by surface reflection, measurement noise, object geometry, and scan availability. For this reason, the LiDAR information is fused with inertial data instead of being used as the only localisation source.

2.3 Inertial Measurement Unit

An inertial measurement unit measures body motion using accelerometers and gyroscopes. The GY-87 module used in this project includes an MPU-6050 accelerometer and gyroscope, an HMC5883L magnetometer, and a BMP180 pressure sensor [3, 2]. For

the indoor ground-vehicle experiment, the important components are the accelerometer, gyroscope, and magnetometer. The pressure sensor is not used for the navigation task.

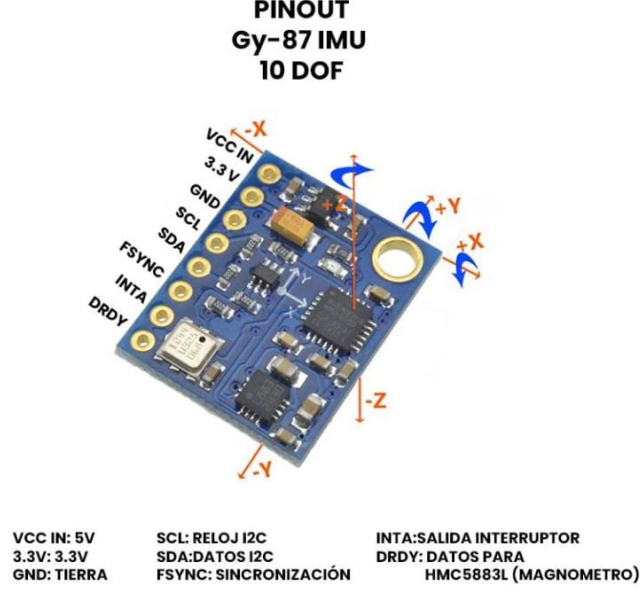


Figure 2.2: GY-87 inertial measurement module used as the onboard motion sensing unit. The relevant signals are read by the Raspberry Pi through the I2C interface.

The accelerometer measures acceleration along its sensor axes. In theory, velocity can be calculated by integrating acceleration over time,

$$v(t) = v(t_0) + \int_{t_0}^t a(\tau) d\tau. \quad (2.2)$$

In practice, this method is not reliable on its own for long-term navigation. Even a small acceleration offset creates a growing velocity error, and after another integration it creates an even larger position error. The gyroscope measures angular velocity and can be used to estimate the yaw angle by integrating the yaw rate,

$$\psi(t) = \psi(t_0) + \int_{t_0}^t \omega_z(\tau) d\tau. \quad (2.3)$$

The gyroscope reacts quickly to turning motion, which makes it important during the right-turn phase of the latest experiment. However, gyroscope bias also causes heading drift. The magnetometer can provide an additional heading reference, but it may be affected by nearby electrical components and magnetic disturbances. Therefore, the sensors must be combined carefully using a filtering method.

2.4 Extended Kalman Filter

The Extended Kalman Filter is a recursive estimation method based on the Kalman filtering principle, which estimates the system state through repeated prediction and measurement correction steps [4]. It is suitable for nonlinear systems because it linearises the nonlinear motion and measurement models around the current estimate. This structure is useful for the robot in this project because inertial data can provide continuous motion prediction, while LiDAR distance information can correct accumulated drift.

The robot state can be represented in the general form

$$\mathbf{x}_k = \begin{bmatrix} x_k & y_k & v_{x,k} & v_{y,k} & \psi_k \end{bmatrix}^T, \quad (2.4)$$

where x_k and y_k are the estimated position coordinates, $v_{x,k}$ and $v_{y,k}$ are the velocity components, and ψ_k is the yaw angle at time step k . The prediction step is written as

$$\hat{\mathbf{x}}_{k|k-1} = f(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_k), \quad (2.5)$$

where $\mathbf{u}_k$ represents the control or inertial input information. The uncertainty of the prediction is propagated by

$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^T + Q_k. \quad (2.6)$$

Here, P is the covariance matrix, F_k is the Jacobian of the motion model, and Q_k is the process noise covariance. The process noise describes uncertainty caused by modelling errors, motor slip, sensor noise, and small timing variations.

When a measurement is available, the Kalman gain is calculated as

$$K_k = P_{k|k-1} H_k^T (H_k P_{k|k-1} H_k^T + R_k)^{-1}. \quad (2.7)$$

The corrected state estimate is then obtained from

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + K_k (\mathbf{z}_k - h(\hat{\mathbf{x}}_{k|k-1})), \quad (2.8)$$

and the covariance is updated by

$$P_{k|k} = (I - K_k H_k) P_{k|k-1}. \quad (2.9)$$

In these equations, H_k is the measurement Jacobian, R_k is the measurement noise covariance, $\mathbf{z}_k$ is the measurement vector, and K_k is the Kalman gain. The Kalman gain determines how much the filter trusts the measurement compared with the prediction. This makes the EKF suitable for the latest implementation, where the robot must combine fast but drifting inertial data with slower but more stable LiDAR distance

correction.

2.5 Embedded Controller and Motor Drive

The Raspberry Pi is used as the embedded controller of the robot and provides GPIO, serial, I2C, and network interfaces suitable for embedded prototyping [11]. It collects sensor data, executes the estimation and control software, generates motor-control signals, logs telemetry, and sends data to the host computer. A Linux-based single-board computer is suitable for this project because Python software, GPIO control, serial communication, I2C communication, and network communication can be implemented on the same platform.

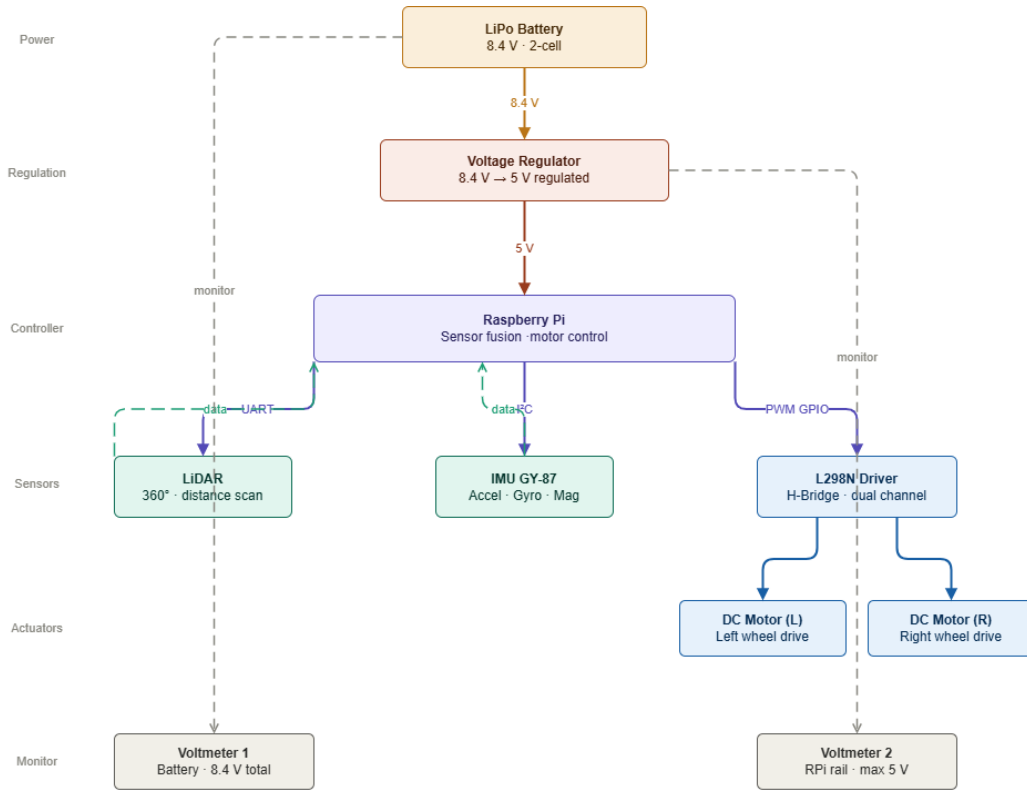


Figure 2.3: System architecture of the implemented robot platform. The Raspberry Pi connects the IMU, LiDAR, motor driver, and UDP telemetry pipeline.

The robot is driven by two DC gear motors using an L298N H-bridge motor driver [14]. An H-bridge allows the polarity across a DC motor to be changed electronically. As a result, each motor can rotate forward or backward depending on the input signals. The speed is controlled by pulse-width modulation, where the effective average voltage is changed by adjusting the duty cycle.

The two motors form a differential-drive system. If both motors rotate forward with similar speed, the robot moves approximately straight. If the wheel speeds are different, the robot turns. The latest experiment uses this principle to perform the sequence

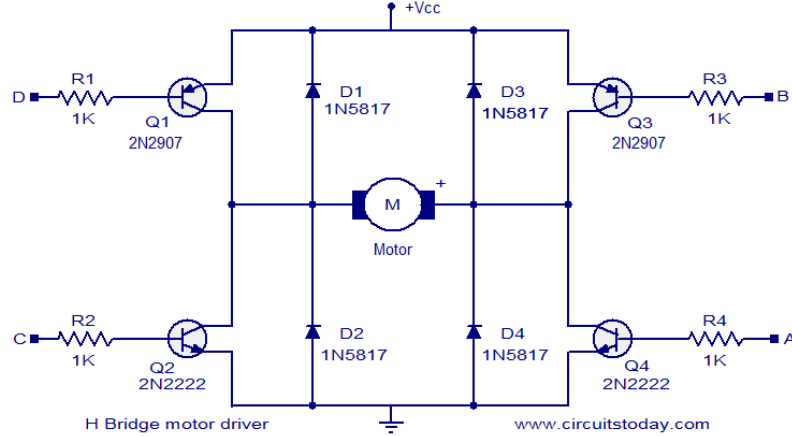


Figure 2.4: Basic H-bridge concept for reversing the polarity across a DC motor. The same principle is used by the L298N motor driver module.

`forward_1`, `turn_right`, and `forward_2`. In practice, the motion is affected by motor imbalance, floor friction, battery voltage, and mechanical tolerances. These effects are one reason why feedback from sensors and EKF-based correction are required.

2.6 UDP Communication, MATLAB Analysis, and RViz Visualisation

The latest implementation uses UDP communication to transmit telemetry from the Raspberry Pi to a host computer. UDP is suitable for live monitoring because it has low communication overhead and can send data continuously without the delay of connection handling. Each telemetry packet contains the current motion phase, time, estimated position, yaw angle, travelled distance, LiDAR distance, velocity information, and motor-control values. These values are needed for both live observation and later evaluation.

MATLAB is used for logging, post-processing, and plotting the received telemetry. The MATLAB results are especially important for the latest half-rectangular test because they show the behaviour during `forward_1`, `turn_right`, and `forward_2`. The plots allow the estimated path, heading, velocity, travelled distance, and LiDAR response to be inspected after the experiment.

ROS 2 and RViz are used as an additional live visualisation pipeline on Ubuntu [8]. The UDP telemetry is converted into ROS 2 messages and displayed in RViz as robot state, path, markers, and transform information. This does not replace the MATLAB evaluation, but it makes the motion behaviour easier to understand during a demonstration. Therefore, the communication pipeline supports both post-experiment

analysis and live visual validation.

2.7 Power Supply and Practical Limitations

The robot is powered from a battery system that supplies both the motors and the Raspberry Pi. The motors require higher current and can introduce voltage drops during acceleration or turning. The Raspberry Pi requires a stable regulated supply, because voltage drops may affect sensor readings, control software, or UDP communication. For this reason, stable power regulation is an important practical part of the robot design.

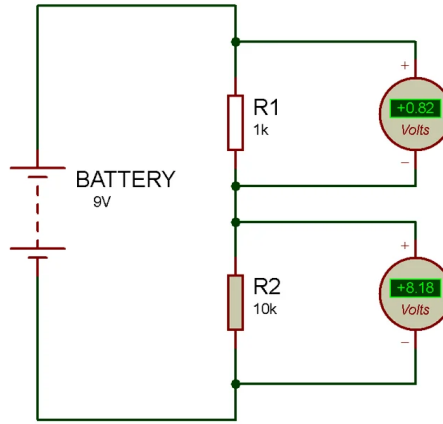


Figure 2.5: Voltage divider principle used for scaling a higher input voltage to a lower measurable voltage range.

A voltage divider can be used to scale a higher input voltage to a lower measurable voltage. The output voltage is

$$V_{\text{out}} = V_{\text{in}} \cdot \frac{R_2}{R_1 + R_2}, \quad (2.10)$$

where V_{in} is the input voltage and R_1 and R_2 are the resistor values. In this project, reliable power is important because the quality of the experiment depends on stable sensor readings, motor behaviour, and communication between the robot and the host computer.

Chapter 3

Related Work

Mobile robot localisation is a central topic in autonomous robotics because a robot must estimate its own position and orientation before it can navigate safely and repeatedly. In many differential-drive mobile robots, this task is performed using wheel encoders. Encoder pulses are converted into wheel rotation, and the wheel rotation is then used to estimate travelled distance and heading change. However, encoder-based odometry is not perfect. Wheel slip, unequal wheel diameters, floor friction, motor imbalance, and mechanical tolerances can cause the estimated motion to differ from the real motion. Borenstein and Feng showed that systematic odometry errors can accumulate significantly in mobile robots if they are not compensated or corrected [1]. This is especially relevant for low-cost robots because small mechanical inaccuracies can have a large influence on the final estimated trajectory.

The localisation problem becomes more challenging when wheel encoders are not used. In an encoder-free robot, the system cannot directly measure wheel rotation, so it must estimate motion from other onboard sensors. This project follows such an encoder-free approach. The latest implementation uses a Raspberry Pi robot with an inertial measurement unit and LiDAR-based distance information. The aim is not to build a high-cost industrial localisation system, but to investigate whether a low-cost embedded platform can still perform structured autonomous motion when sensor fusion and external visualisation tools are used. This idea is consistent with the general principles of mobile robot localisation described by Siegwart, Nourbakhsh, and Scaramuzza, where reliable navigation normally requires combining a motion model with sensor observations [12].

Probabilistic robotics provides an important theoretical background for this work. Thrun, Burgard, and Fox describe robot localisation as a problem of estimating a belief over the robot state under uncertainty [16]. This is important because real sensors and actuators are never ideal. IMU data contain noise and bias, LiDAR measurements can be affected by reflections and environmental geometry, and DC motors can behave differently depending on load, surface, and battery voltage. Instead of assuming that

the measured values are perfect, probabilistic estimation methods explicitly consider uncertainty. This is the reason why a filtering method is suitable for the present project: the robot state is not measured directly, but estimated from imperfect sensor data.

The Kalman filter is one of the most widely used recursive methods for state estimation. Kalman introduced the filter as a mathematical method for estimating the state of a dynamic system from noisy measurements [4]. Welch and Bishop later explained the Kalman filter as a practical prediction-correction framework for tracking and navigation problems [18]. The basic idea is that the filter first predicts the next state using a model, and then corrects this prediction when a new measurement is available. For nonlinear systems, such as a mobile robot that changes position and yaw during turning, the Extended Kalman Filter is commonly used. It linearises the nonlinear model around the current estimate and then applies the Kalman update. This makes the EKF suitable for the latest implementation, where the robot state includes position, velocity, and yaw angle.

Inertial measurement units are often used for short-term motion estimation. A gyroscope measures angular velocity and is useful for detecting rotation, especially during turning. An accelerometer measures acceleration and can in principle be integrated to estimate velocity and position. However, inertial navigation is strongly affected by drift. Titterton and Weston explain that small sensor biases in inertial systems accumulate over time and can lead to large errors if they are not corrected [17]. This limitation is directly relevant to the GY-87 module used in this project. The gyroscope can capture the right-turn motion quickly, but the yaw estimate can drift. The accelerometer can provide motion information, but pure double integration is not reliable for accurate long-term position estimation on a low-cost mobile robot.

For this reason, several works use filtering methods to improve IMU-based orientation estimation. Mahony, Hamel, and Pfimlin proposed nonlinear complementary filters for attitude estimation, while Madgwick presented an efficient orientation filter for inertial and magnetic sensor arrays [7, 6]. These approaches show that IMU data are useful when they are processed carefully and combined with other references. In this project, the IMU is not treated as a complete localisation solution by itself. Instead, it provides fast heading and motion information that is corrected using additional sensor information.

LiDAR is a common sensing technology in mobile robotics because it provides distance information about the surrounding environment. Unlike inertial integration, LiDAR distance measurements do not accumulate drift in the same way. Leonard and Durrant-Whyte showed that range information and geometric features can be used as references for mobile robot localisation [5]. In the present project, the LiDAR is used in a simpler and more practical way. It is not used for full simultaneous localisation and mapping. Instead, it provides local distance information to a wall or nearby object.

This distance information can be used as a correction source for the estimated position during structured indoor motion. This makes the LiDAR an important support sensor for reducing the drift that would occur with IMU-only estimation.

The combination of IMU and LiDAR is therefore well motivated by the literature. The IMU provides fast dynamic information but suffers from drift, while LiDAR provides more stable distance information but depends on the environment and scan quality. Sensor fusion combines these complementary properties. In the latest implementation, the EKF acts as the fusion method that connects the prediction from inertial data with correction from LiDAR-based distance measurements. This follows the general principle of probabilistic localisation: the robot prediction is continuously improved when a useful external measurement becomes available.

The robot platform itself is also related to work on differential-drive mobile robots. A differential-drive robot moves by controlling the left and right wheel speeds independently. When both wheels move forward with similar speed, the robot travels approximately straight. When the wheel speeds differ, the robot turns. This motion principle is simple and widely used, but it is sensitive to motor mismatch and surface friction. Without wheel encoders, the motor command does not guarantee the exact travelled distance. Therefore, a robot using open-loop PWM motor control requires careful calibration and sensor-based validation. Borenstein and Feng also showed that differential-drive errors can come from both systematic and non-systematic sources, which supports the need for correction and evaluation in this project [1].

Open-loop PWM control is often used in low-cost embedded robots because it is simple and easy to implement with GPIO pins and an H-bridge driver. A PWM signal changes the effective average motor voltage by changing the duty cycle. However, the real wheel speed depends on battery voltage, motor load, friction, and mechanical differences between the left and right motors. This means that PWM control alone cannot guarantee exact motion. In the latest implementation, the L298N motor driver and Raspberry Pi GPIO signals provide practical motor control, but the accuracy of the motion must be evaluated using the recorded telemetry and sensor estimates rather than assumed from the PWM values alone.

Another relevant part of the project is the use of external analysis and visualisation tools. MATLAB is widely used in engineering projects for data logging, post-processing, and plotting. In this work, MATLAB is used to analyse the received robot telemetry after the experiment. The plotted data help to understand the estimated path, travelled distance, heading behaviour, velocity, and LiDAR response. ROS is also an important framework in robotics. Quigley et al. introduced ROS as an open-source robot operating system for building robot software components [9]. In this project, ROS 2 and RViz are used as a visualisation pipeline. The UDP telemetry from the Raspberry Pi is converted into ROS 2 messages so that the robot state, path, markers,

and transforms can be observed during operation. This supports demonstration and validation, even though the numerical evaluation is mainly performed using the logged MATLAB data.

The literature also shows that the evaluation of a robot must be linked to the actual experiment that was performed. Therefore, the latest practical test in this report is described as an L-shaped, half-rectangular motion sequence. The robot performed a first forward movement, a right-angle turn, and a second forward movement. These are the basic motion primitives required for a complete rectangular trajectory. A full rectangular path can be created by repeating straight-line movements and right-angle turns. However, it would not be correct to claim a complete rectangular experiment unless all four sides were actually tested and recorded. For this reason, the full rectangular motion is treated in this report as an applicable extension of the implemented method, while the latest real experimental evidence is limited to the half-rectangular test.

Overall, the related work supports the main design decision of this project: a low-cost, encoder-free mobile robot can improve its motion estimation by combining inertial prediction with LiDAR-based correction. Previous research on odometry errors, probabilistic localisation, Kalman filtering, inertial orientation estimation, LiDAR-based localisation, and differential-drive robot motion all point to the same conclusion: no single low-cost sensor is sufficient for reliable navigation, but a carefully designed sensor-fusion and validation pipeline can produce a more stable estimate. The following chapters build on this background by analysing the requirements, describing the implemented Raspberry Pi, UDP, MATLAB, and ROS 2/RViz workflow, and evaluating the latest L-shaped experimental data.

Chapter 4

Analysis

The system is designed around one central constraint: the robot must estimate its movement without wheel encoders. This constraint affects the sensor selection, movement strategy, communication structure, and evaluation method. Since motor commands cannot be converted directly into exact travelled distance, the system requires sensor-based correction. In the latest implementation, LiDAR distance changes are used as the main correction source during straight motion, while the IMU supports heading estimation and turn detection. The EKF combines these measurements so that the robot state can be estimated more reliably than with one sensor alone [4, 18].

The latest physical experiment is not a complete rectangular route. It is an L-shaped, half-rectangular motion sequence consisting of a first forward movement, one right turn, and a second forward movement. The recorded phases in the latest telemetry file are `forward_1`, `turn_right`, and `forward_2`. The first forward segment has a target distance of 300 mm, the turn has a target heading change of approximately -90° , and the second forward segment has a target distance of 200 mm. This route contains the two basic movement primitives required for a full rectangular trajectory: straight-line movement and right-angle turning. Therefore, the method is applicable to a full rectangular motion, but the completed four-sided rectangle is not presented as a measured result in this report.

4.1 System Requirements

The main functional requirement is that the robot must move through a bounded indoor path while recording enough telemetry to evaluate the motion afterwards. Because the vehicle does not use wheel encoders, the evaluation cannot rely on wheel rotation data. Instead, the robot must record the active motion phase, target distance, estimated distance, LiDAR distance, yaw angle, and PWM motor commands. These signals are necessary because they allow the experiment to be analysed from both the motion-control side and the sensor-estimation side.

The Raspberry Pi must perform the control and processing tasks on the robot. These tasks include reading the IMU and LiDAR data, executing the motion sequence, calculating the EKF-based estimate, controlling the motors through the L298N driver, and sending telemetry to the host computer. The host computer must then log and visualise the data. MATLAB is used for CSV-based post-processing and plot generation, while ROS 2 and RViz are used for live visualisation on Ubuntu. This separation allows the Raspberry Pi to remain focused on embedded control, while the more powerful host systems handle analysis and display [10, 9].

The communication requirement is that the telemetry should be transmitted in a simple and lightweight form. UDP is used for this purpose because it allows the Raspberry Pi to send live data continuously without the overhead of a heavy middleware stack running directly on the robot. The latest telemetry contains the following fields: time, phase, target segment distance, time-based distance, raw LiDAR distance, filtered LiDAR distance, LiDAR segment travel, EKF distance, used distance, set yaw, actual yaw, left PWM, and right PWM. These fields form the basis for the MATLAB analysis and the RViz visualisation.

4.2 Latest Raspberry Pi to MATLAB Data

The first data source is the real Raspberry Pi experiment. In this workflow, the robot sends telemetry directly from the Raspberry Pi to the host computer through UDP. The received data are then saved and plotted in MATLAB. This dataset represents the main physical experiment of the report and is used to evaluate the latest half-rectangular motion.

The latest Raspberry Pi dataset covers the complete L-shaped test sequence. The recorded data confirm that the physical experiment consists of three phases only. During `forward_1`, the target segment distance is 300 mm. During `turn_right`, the robot rotates toward approximately -90° . During `forward_2`, the target segment distance is 200 mm. No fourth segment, return segment, or closed-loop rectangular trajectory is included in the latest real data.

Table 4.1: Latest recorded motion phases from the real Raspberry Pi to MATLAB UDP experiment.

Phase	Target	Final EKF	Final yaw	PWM L/R
<code>forward_1</code>	300 mm	300.47 mm	9.94°	50 / 60
<code>turn_right</code>	-90°	0.00 mm	-85.34°	60 / 60
<code>forward_2</code>	200 mm	200.86 mm	-89.99°	50 / 64

Table 4.1 summarises the latest recorded motion phases. The EKF distance estimate reaches approximately 300.47 mm for the first forward segment and approximately

200.86 mm for the second forward segment. The right-turn phase reaches approximately -85.34° at the end of the turning phase, while the yaw later settles near -89.99° during the second forward movement. This means that the latest data are suitable for evaluating the L-shaped trajectory, but not for evaluating a full return-to-origin rectangle.

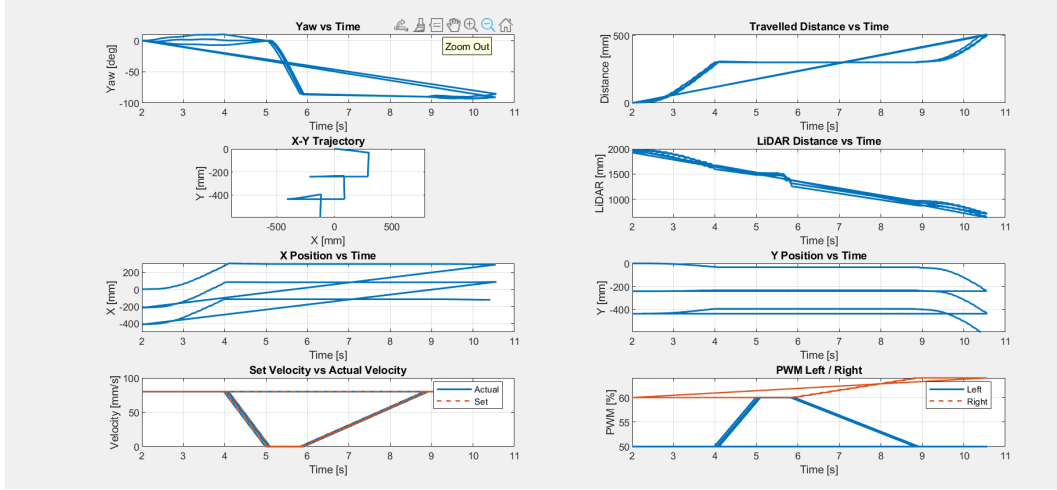


Figure 4.1: Real Raspberry Pi to MATLAB dashboard. The telemetry was transmitted from the Raspberry Pi through UDP and plotted in MATLAB. This figure represents the latest physical half-rectangular experiment.

Figure 4.1 is used as the main overview of the real experimental data. It belongs to the Raspberry Pi to MATLAB path and should not be confused with the Ubuntu/ROS 2/RViz visualisation data.

4.3 Ubuntu, ROS2/RViz, and MATLAB Visualisation Data

The second data source is the Ubuntu visualisation workflow. In this workflow, the UDP telemetry is received and converted into ROS 2 messages. RViz is used to display the robot state, path, and visual markers during operation. The same visualisation-related data can also be plotted afterwards in MATLAB. Therefore, Figures 4.2 to 4.8 are treated as Ubuntu/ROS 2/RViz-based visualisation data that were later plotted in MATLAB. They are useful for checking the live visualisation pipeline and for explaining the behaviour of the telemetry signals.

Figure 4.2 shows the yaw error during the recorded sequence. The large peak corresponds to the turning interval, where the set yaw changes toward the right-angle target and the actual yaw follows with a delay. This figure is useful for understanding the transient heading error during the right-turn phase.

Figure 4.3 shows the difference between the used distance and the EKF distance. This is important because the robot is encoder-free, and the distance estimate is gen-

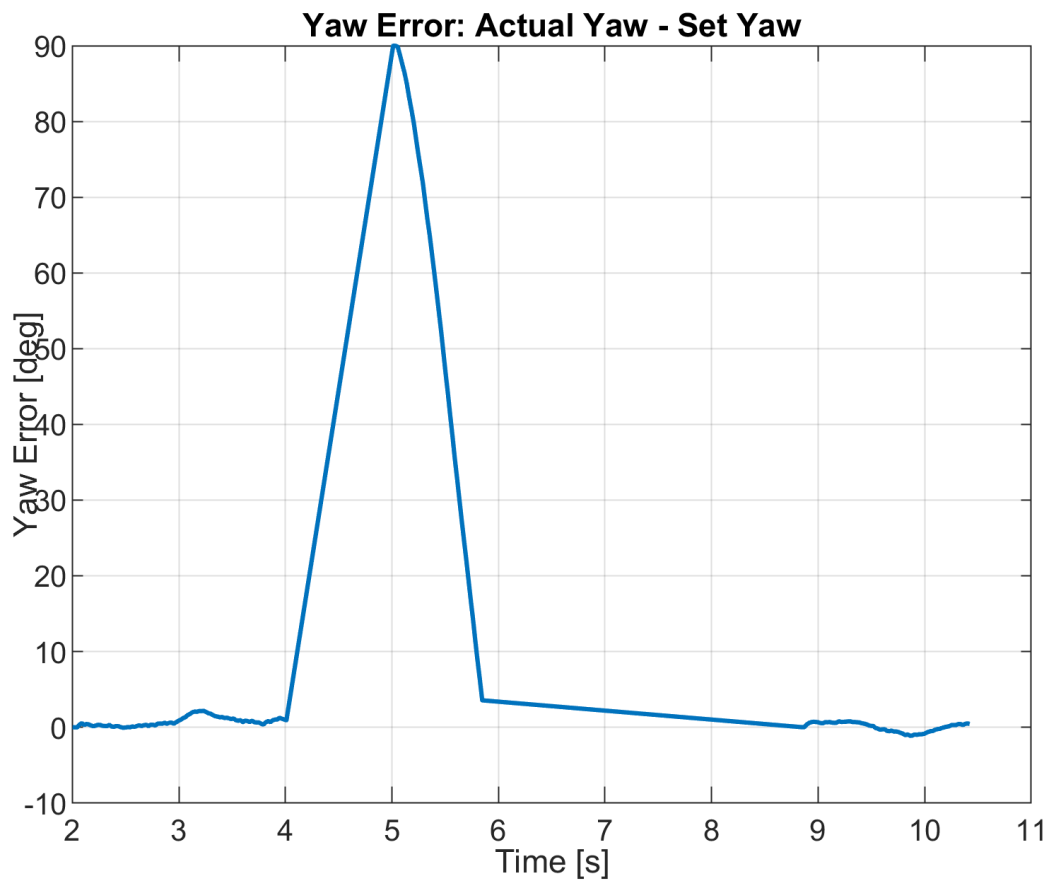


Figure 4.2: Yaw-error plot from the Ubuntu/ROS2/RViz data later plotted in MATLAB. The graph shows the difference between actual yaw and set yaw during the half-rectangular motion.

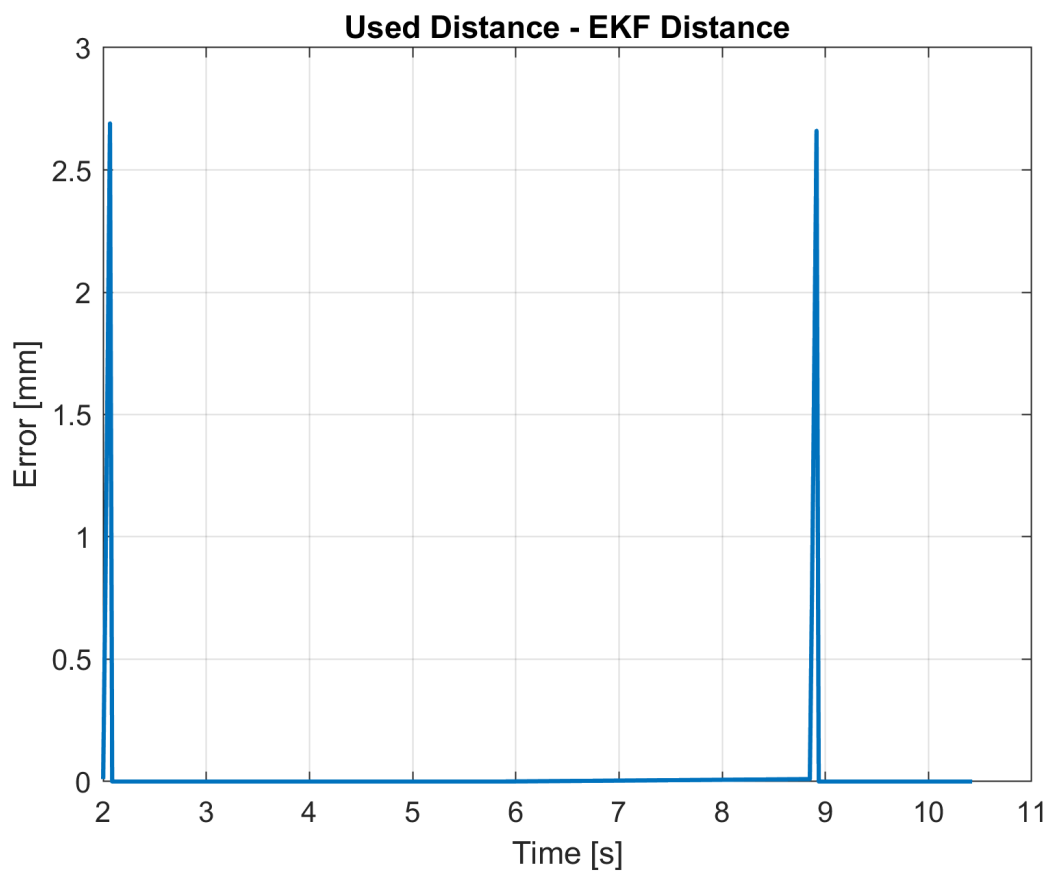


Figure 4.3: Distance-estimation error from the Ubuntu/ROS2/RViz data later plotted in MATLAB. The graph shows the difference between used distance and EKF distance.

erated from sensor-based estimation rather than direct wheel rotation. The plot helps identify the moments where the selected distance signal differs from the EKF estimate.

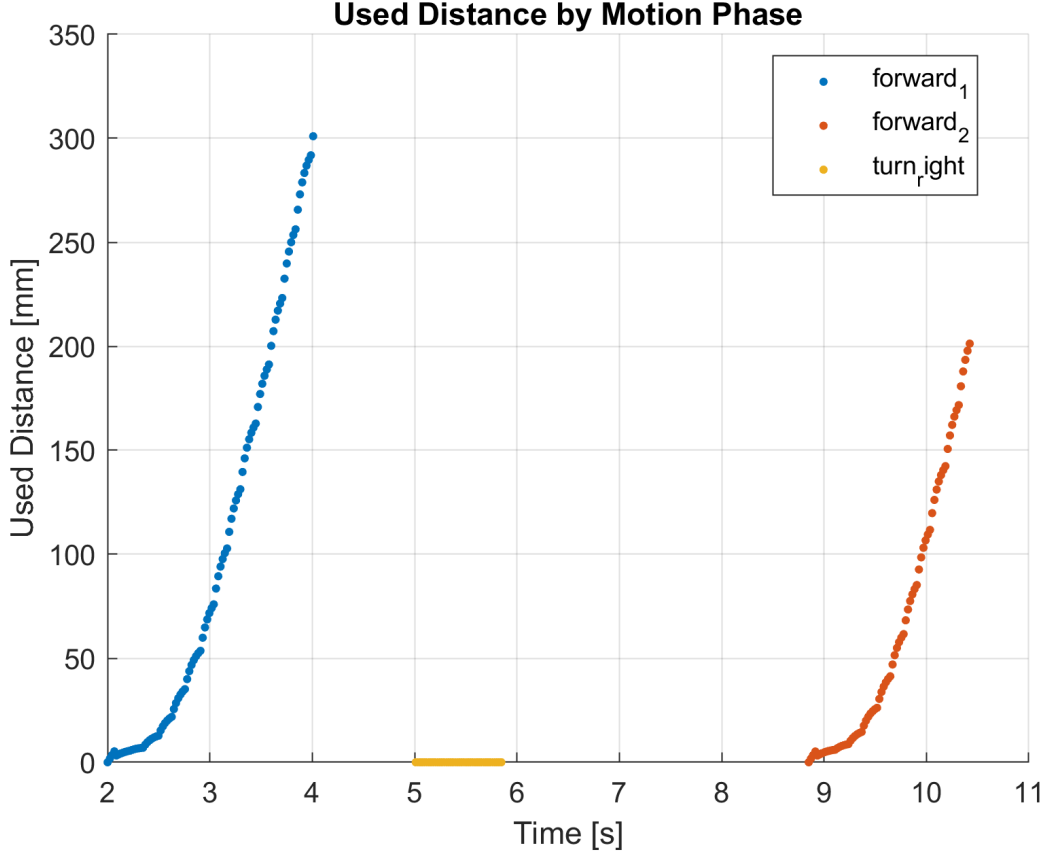


Figure 4.4: Used distance by motion phase from the Ubuntu/ROS2/RViz data later plotted in MATLAB. The graph separates the distance behaviour for `forward_1`, `turn_right`, and `forward_2`.

Figure 4.4 shows how the used distance changes during each motion phase. The first forward phase reaches approximately 300 mm, the turn phase remains close to zero distance, and the second forward phase reaches approximately 200 mm. This confirms that the latest dataset represents an L-shaped half-rectangular experiment.

Figure 4.5 presents the PWM commands sent to the left and right motors. These values are useful for explaining the control strategy. During the first forward motion, the right motor uses a higher PWM value than the left motor to compensate for mechanical imbalance. During the turning phase and the second forward phase, the motor commands change according to the intended motion.

Figure 4.6 shows the yaw behaviour. This figure is important for checking whether the robot turns toward the intended right-angle orientation and whether the heading remains close to the target during the second forward phase.

Figure 4.7 shows the LiDAR-related behaviour. The raw LiDAR signal contains measurement fluctuations, while the Kalman-filtered LiDAR signal provides a smoother distance estimate. This supports the use of filtering before the distance information is

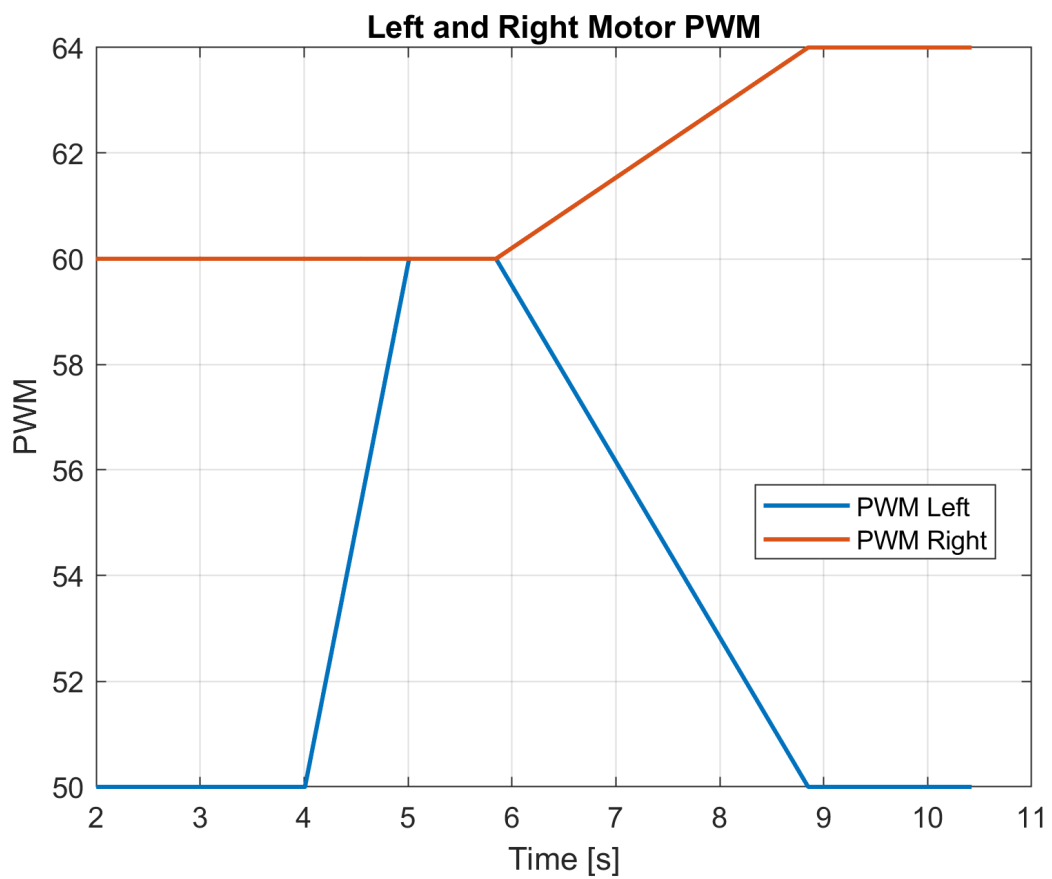


Figure 4.5: Left and right motor PWM commands from the Ubuntu/ROS2/RViz data later plotted in MATLAB. The graph shows the phase-dependent motor-command values used during the recorded motion.

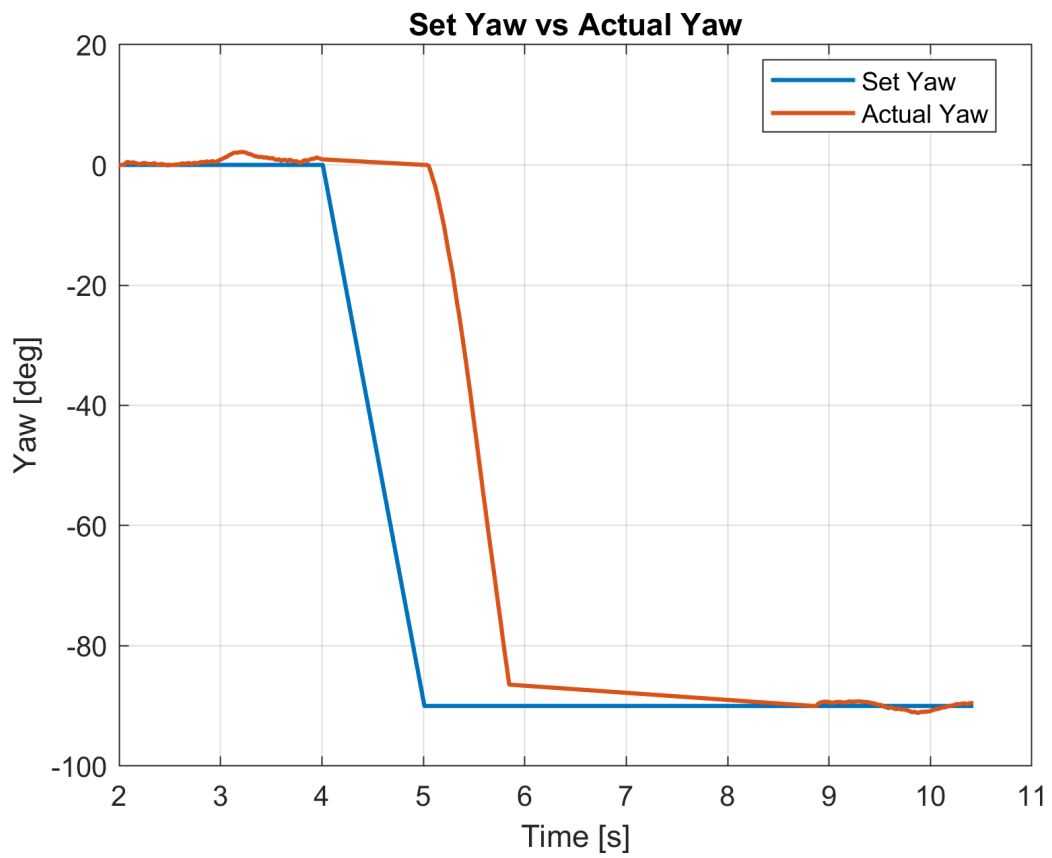


Figure 4.6: Set yaw and actual yaw from the Ubuntu/ROS2/RViz data later plotted in MATLAB. The graph shows the yaw response during the right-turn phase and the second forward movement.

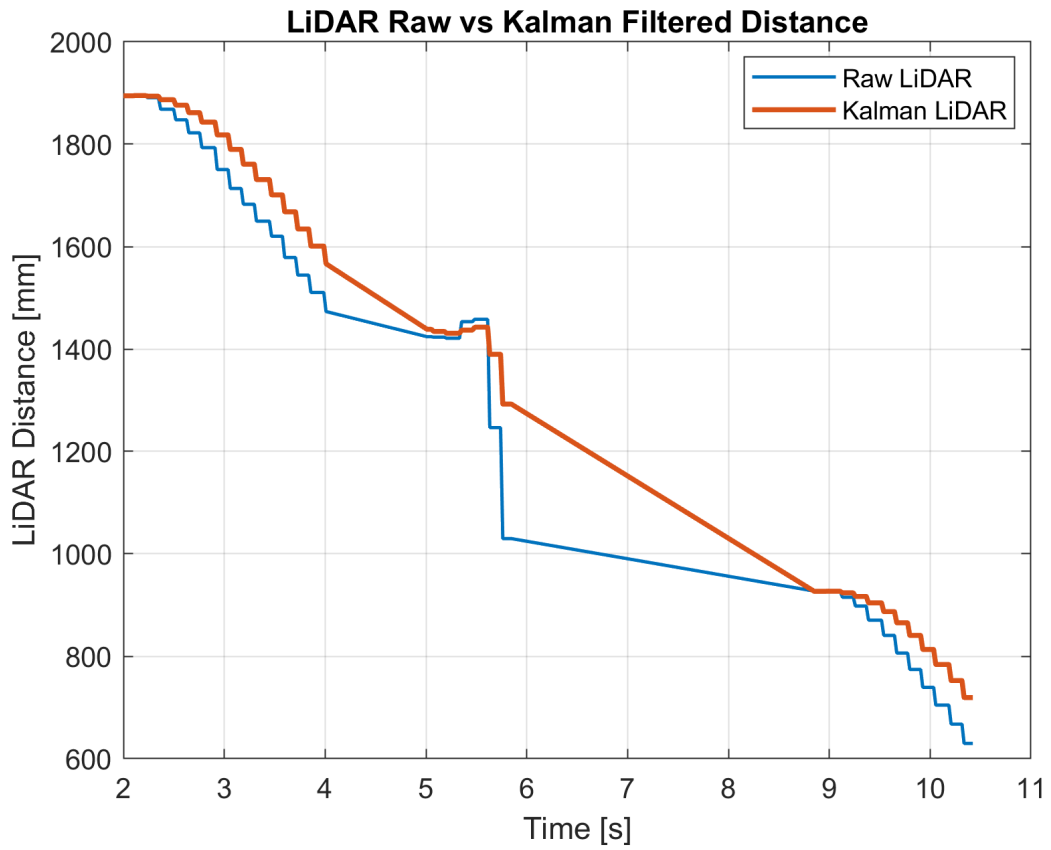


Figure 4.7: Raw LiDAR and Kalman-filtered LiDAR distance from the Ubuntu/ROS2/Rviz data later plotted in MATLAB. The graph compares the raw ranging signal with the filtered distance estimate.

interpreted for motion estimation.

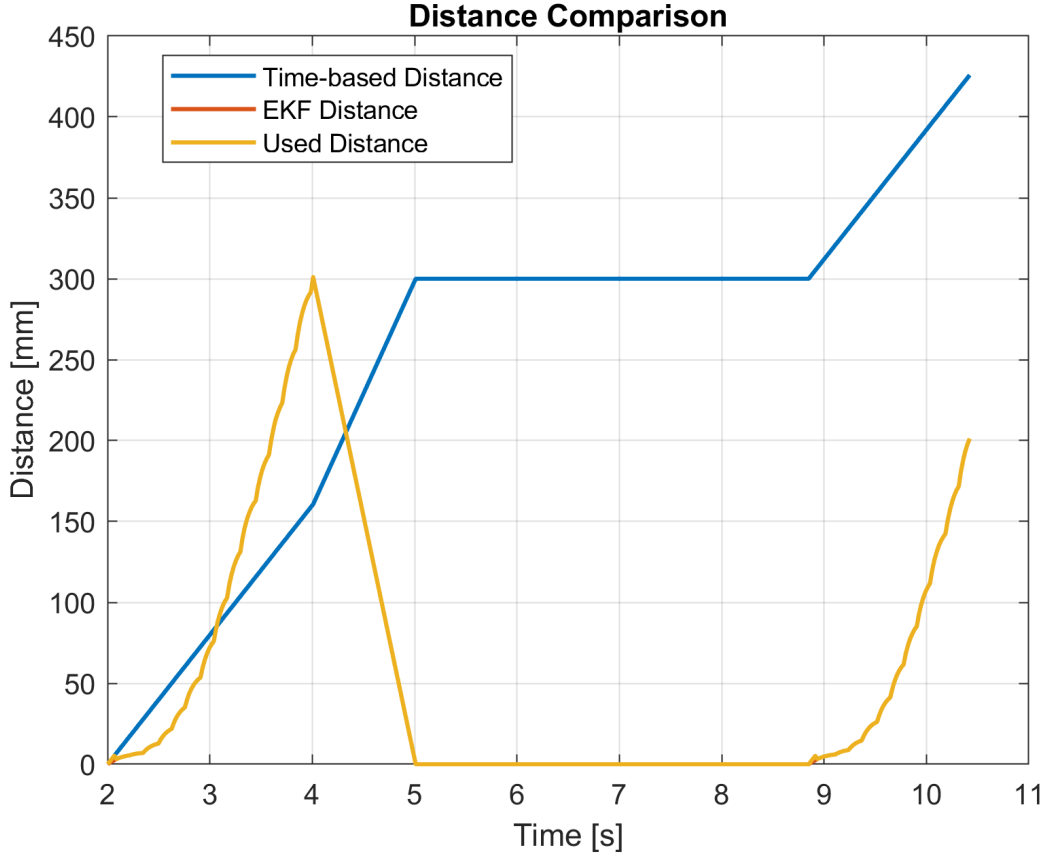


Figure 4.8: Distance comparison from the Ubuntu/ROS2/RViz data later plotted in MATLAB. The graph compares time-based distance, EKF distance, and used distance during the half-rectangular test.

Figure 4.8 summarises the main distance signals used in the visualisation workflow. It compares the time-based distance, the EKF distance, and the used distance. The plot should be interpreted only as a summary of the L-shaped experiment and not as evidence of a completed full rectangular path.

4.4 Technical Constraints

The most important technical limitation is the absence of wheel encoders. Motor output depends on battery voltage, surface friction, robot load, and mechanical differences between the left and right sides of the drive system. Even if the PWM duty cycle is fixed, the travelled distance can vary. For this reason, PWM values alone cannot prove that the robot travelled the commanded distance. The recorded sensor estimates and telemetry are required to evaluate the real motion. This agrees with known limitations of differential-drive odometry and motor mismatch in mobile robots [1, 12].

The LiDAR helps to correct the drift, but it also has limitations. It depends on visible reference surfaces and can produce noisy readings when the environment is

not ideal. The latest data include raw LiDAR distance, filtered LiDAR distance, and LiDAR segment travel. These values are important because they show how the ranging information changes during the movement. However, LiDAR alone is not enough for complete navigation, because it does not directly provide a full robot pose in every situation.

The IMU provides fast heading information and is especially important for turn detection. However, low-cost inertial sensors are affected by bias, noise, and magnetic disturbances. The yaw estimate can therefore drift if it is not corrected. The EKF is used to combine the useful parts of the available measurements and reduce the effect of individual sensor weaknesses. This is the main reason why the project is analysed as a sensor-fusion system rather than as a simple motor-control experiment.

Communication is another practical constraint. Running all visualisation tools directly on the Raspberry Pi would make the embedded system more complex and could reduce processing performance. Therefore, the Raspberry Pi sends compact UDP telemetry to the host computer. MATLAB and ROS 2/RViz then run externally. This design keeps the robot software lightweight while still allowing detailed analysis and live visualisation.

4.5 Evaluation Criteria

The evaluation criteria are selected according to the data that was actually recorded in the latest experiment. The first criterion is the segment-distance accuracy of the 300 mm and 200 mm forward movements. The second criterion is the yaw behaviour during and after the right turn. The third criterion is the relationship between raw LiDAR, filtered LiDAR, LiDAR segment travel, and EKF distance. The fourth criterion is the UDP data path, because the same telemetry must support MATLAB logging and RViz visualisation. The fifth criterion is the comparison between EKF-based estimation and no-EKF behaviour using the simulation results included in the project files.

The evaluation does not use return-to-origin error, full perimeter error, or repeatability over ten complete rectangular runs. These would be suitable metrics for a completed full rectangular experiment, but they require raw data from all four sides of the rectangle. Since the latest physical data only contains `forward_1`, `turn_right`, and `forward_2`, using full-rectangle metrics would overstate the real experimental evidence. For this reason, the full rectangular trajectory is discussed as an extension of the implemented method rather than as a completed result.

4.6 Data and Visualisation Workflow

The project uses two host-side workflows. The first workflow is the Raspberry Pi to MATLAB analysis path. In this path, the Raspberry Pi sends real telemetry over UDP, and the host computer records and plots the received data in MATLAB. This workflow is the main numerical basis for the real experiment.

The second workflow is the Ubuntu/ROS 2/RViz visualisation path. In this path, UDP telemetry is received on Ubuntu and converted into ROS 2 messages. RViz is used to display the robot state, path, and visual markers. The same visualisation data can be plotted later in MATLAB, which is why Figures 1–7 are included separately instead of using one combined screenshot. This workflow is useful for live demonstration because it allows the motion behaviour to be observed while the robot is running.

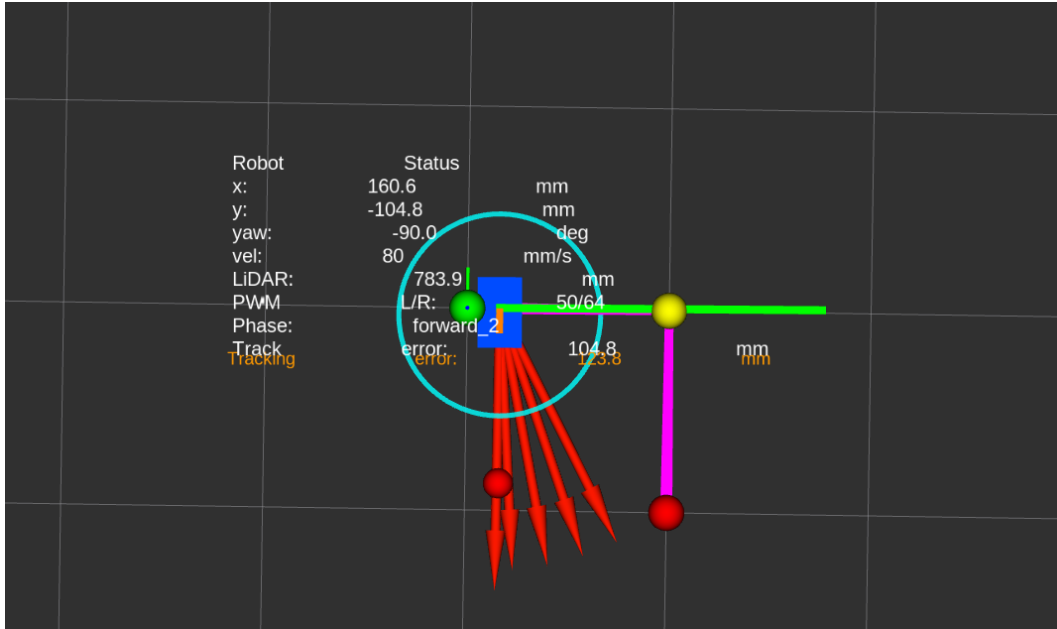


Figure 4.9: RViz live visualisation of the robot telemetry received through the UDP-to-ROS2 pipeline.

Figure 4.9 shows the live RViz view of the robot telemetry. This figure belongs to the Ubuntu/ROS 2/RViz workflow and supports the demonstration of live state visualisation. It is separate from the Raspberry Pi to MATLAB dashboard, which is used as the main physical data overview.

The analysis therefore focuses on the latest available evidence: the real Raspberry Pi to MATLAB UDP data, the Ubuntu/ROS 2/RViz visualisation data later plotted in MATLAB, the RViz live visualisation, and the EKF versus no-EKF simulation comparison. This keeps the report consistent with the actual project status and avoids unsupported claims about a completed full rectangular test.

Chapter 5

Design and Implementation

This chapter describes the physical and software implementation of the autonomous navigation system. The focus is on the realised robot platform, the electrical wiring, the sensing and actuation structure, the EKF-based estimation workflow, and the UDP-based communication pipeline. The mechanical and circuit design from the original implementation are retained because they describe the actual hardware platform. The experimental data and motion sequence, however, are updated to match the latest available test: an L-shaped, half-rectangular motion consisting of `forward_1`, `turn_right`, and `forward_2`. The completed four-sided rectangle is treated as a future extension and is not presented as a measured result in this chapter.

5.1 System Architecture

The autonomous navigation car is organised around four main subsystems: sensing, processing, actuation, and power supply. The sensing subsystem includes the rotating two-dimensional LiDAR sensor and the GY-87 inertial measurement unit. The LiDAR provides distance information to nearby objects or walls, while the IMU provides acceleration, angular-rate, and magnetic heading information. These measurements are required because the vehicle does not use wheel encoders. As a result, the robot cannot directly measure wheel rotation and must estimate its motion from onboard sensors.

The processing subsystem is the Raspberry Pi single-board computer. It reads the sensor data, executes the estimation and motion-control software, generates motor-control signals, logs telemetry, and sends live data to external host systems. The Raspberry Pi was selected because it provides GPIO, serial, I2C, and network interfaces on one compact embedded platform [10]. The actuation subsystem consists of an L298N dual H-bridge motor driver and two DC gear motors. The driver receives GPIO direction commands and PWM enable signals from the Raspberry Pi and controls the left and right motors independently [15].

The power subsystem supplies the motors and embedded electronics. The motor

driver is powered from the main battery path, while the Raspberry Pi requires a regulated 5 V supply. A common ground is used between the battery, L298N driver, voltage regulator, Raspberry Pi, and sensors so that all digital control signals have the same electrical reference.

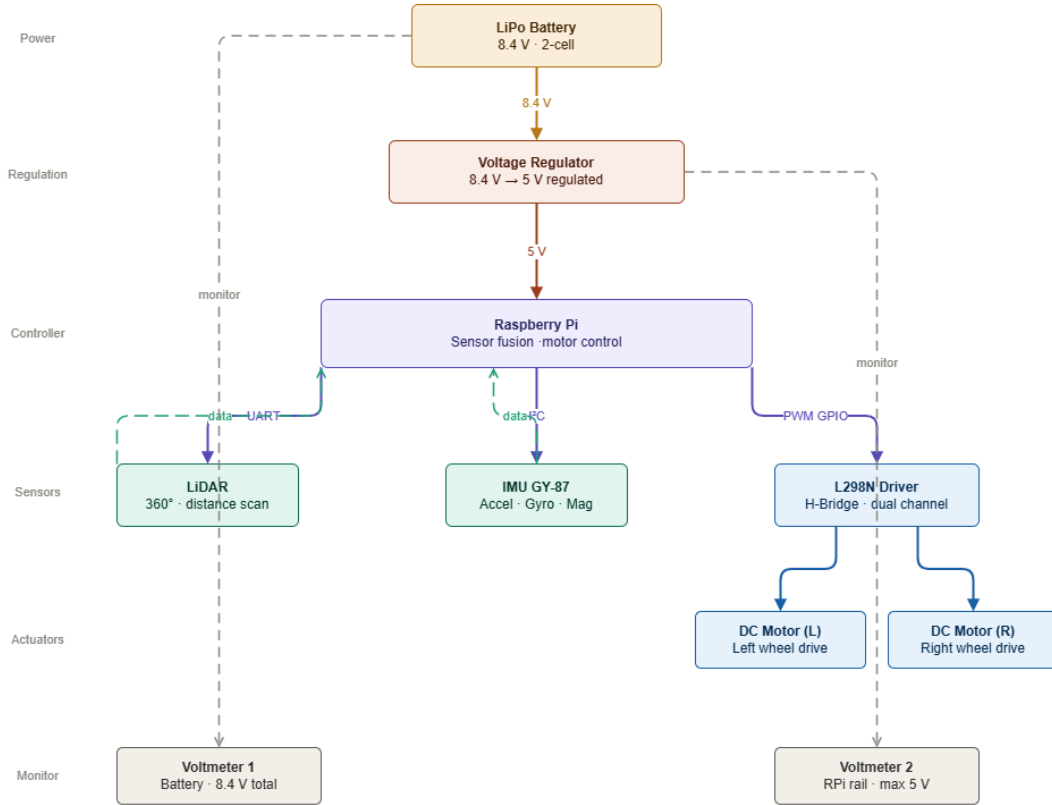


Figure 5.1: Overall system architecture of the autonomous navigation vehicle. The Raspberry Pi is the central processing unit connecting the LiDAR, IMU, motor driver, power subsystem, and UDP telemetry output.

Figure 5.1 shows the implemented architecture. The Raspberry Pi receives sensor data, performs state estimation and control, and transmits telemetry to the external analysis and visualisation systems. In the latest workflow, the UDP telemetry is used in two ways: first for direct Raspberry Pi to MATLAB logging, and second for Ubuntu/ROS2/RViz live visualisation.

5.2 Mechanical Platform Design

The vehicle is built on a compact acrylic chassis that provides a rigid mounting surface for the motors, Raspberry Pi, LiDAR, motor driver, battery, and wiring. The mechanical design is intended to keep the centre of gravity low and to provide a stable mounting position for the sensors. A differential-drive layout is used, with two driven rear wheels and a passive front castor wheel. This arrangement allows the robot to

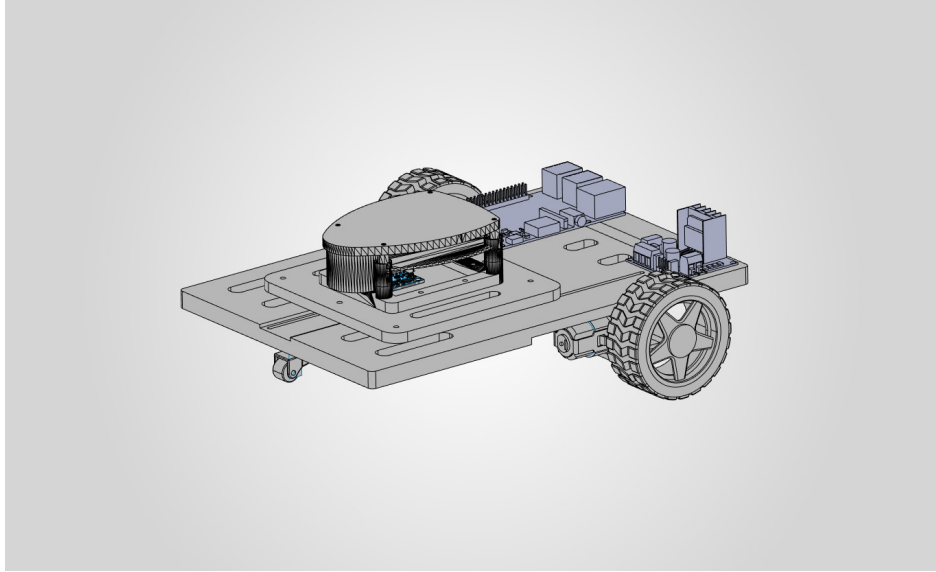


Figure 5.3: Isometric view of the chassis assembly with mounting positions for the sensors, motors, and electronic components.

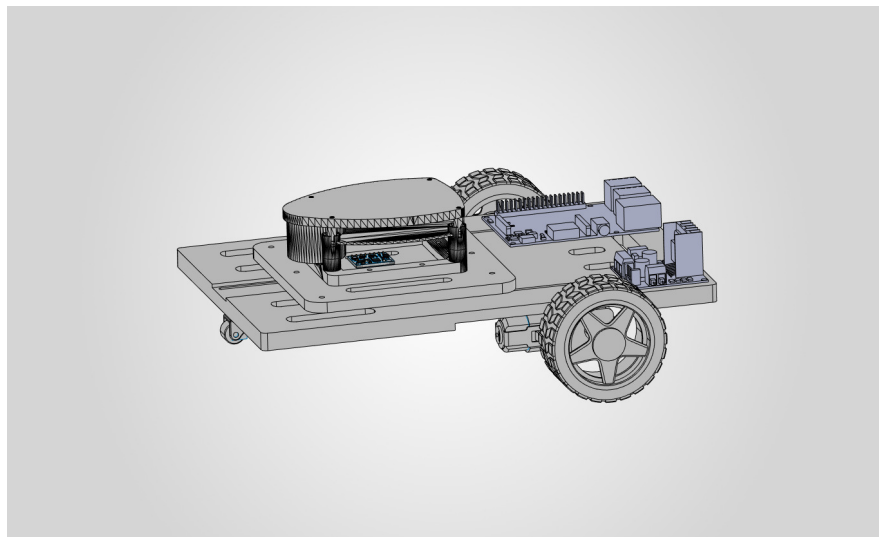


Figure 5.4: Assembled autonomous navigation vehicle showing the complete mechanical and electrical integration.

pins provide the digital direction and PWM control signals for the L298N driver. The LiDAR communicates through a serial interface, while the IMU is connected through the I2C bus. The system uses a common ground to ensure reliable logic-level communication between the Raspberry Pi and the motor driver.

The L298N driver receives direction commands for both motors. The Raspberry Pi sets the motor direction using GPIO output pins and controls motor speed using PWM enable signals. Table 5.1 summarises the main GPIO connections used for motor control and voltage monitoring.

Table 5.1: Raspberry Pi GPIO pin assignments used for motor control and voltage monitoring.

GPIO / Channel	Function	Connected to
GPIO 17	Left motor direction input 1	L298N IN1
GPIO 27	Left motor direction input 2	L298N IN2
GPIO 18	Left motor PWM enable	L298N ENA
GPIO 22	Right motor direction input 1	L298N IN3
GPIO 23	Right motor direction input 2	L298N IN4
GPIO 13	Right motor PWM enable	L298N ENB
ADC Ch 0	Battery voltage monitor	Voltage divider output
ADC Ch 1	Raspberry Pi supply monitor	Voltage divider output

The voltage monitoring circuit is used to observe the battery and regulated supply rails. A voltage divider scales the measured voltage into a range that can be read safely by the monitoring circuit. This is important because motor startup and turning can create voltage drops, and the Raspberry Pi requires a stable supply for reliable sensor reading, control execution, and UDP communication.

The circuit diagram and vehicle front view in Figures 5.5 and 5.6 document the actual hardware integration. These figures are kept in the report because they explain the physical construction of the system and are independent of whether the latest experiment is a half-rectangular or full-rectangular trajectory.

5.4 Software and Communication Architecture

The navigation software is implemented in Python and is divided into sensor reading, estimation, motor control, logging, and communication tasks. The LiDAR driver reads scan data and extracts distance information relevant to the robot motion. The IMU driver reads accelerometer, gyroscope, and magnetometer data from the GY-87 module. The estimation module applies the EKF logic, while the motor-control module sends GPIO and PWM commands to the L298N driver.

The software is organised so that sensor acquisition does not block the motion control loop. LiDAR and IMU reading are handled continuously, while the main navi-

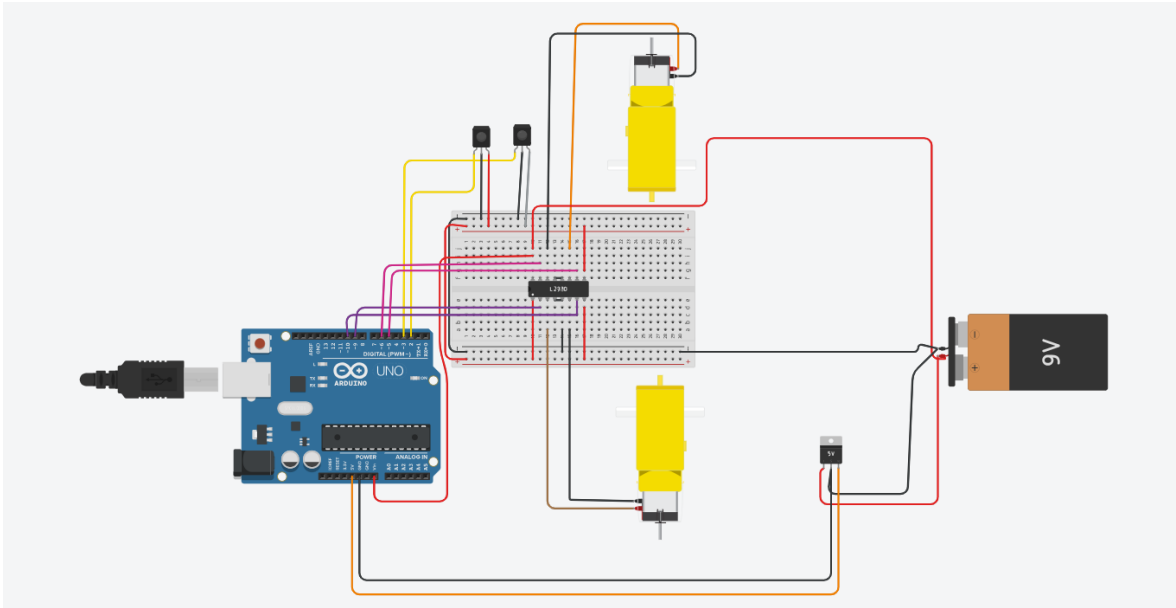


Figure 5.5: Electrical wiring diagram of the robot platform showing the power distribution, L298N motor-driver connections, Raspberry Pi interface, and sensor communication paths.

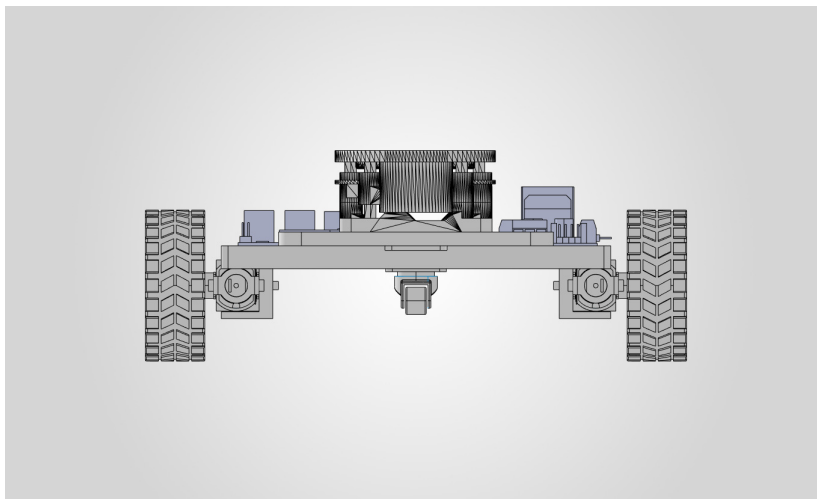


Figure 5.6: Front view of the vehicle showing the LiDAR mounting position, wiring, and sensor integration on the robot platform.

gation loop evaluates the current phase of the motion sequence and sends the required motor commands. This structure is necessary because the robot must read sensors, update the estimate, control the motors, and transmit telemetry at the same time.

The latest communication architecture uses UDP telemetry. The Raspberry Pi sends the current time, motion phase, target distance, estimated distance, LiDAR information, yaw angle, and PWM values to an external computer. This telemetry is used in two workflows. In the first workflow, the data are received directly by MATLAB and saved for plotting and analysis. In the second workflow, the UDP data are received on Ubuntu, converted into ROS2 messages, and visualised in RViz. This design keeps the embedded Raspberry Pi software lightweight while allowing more detailed processing and visualisation on the host computer.

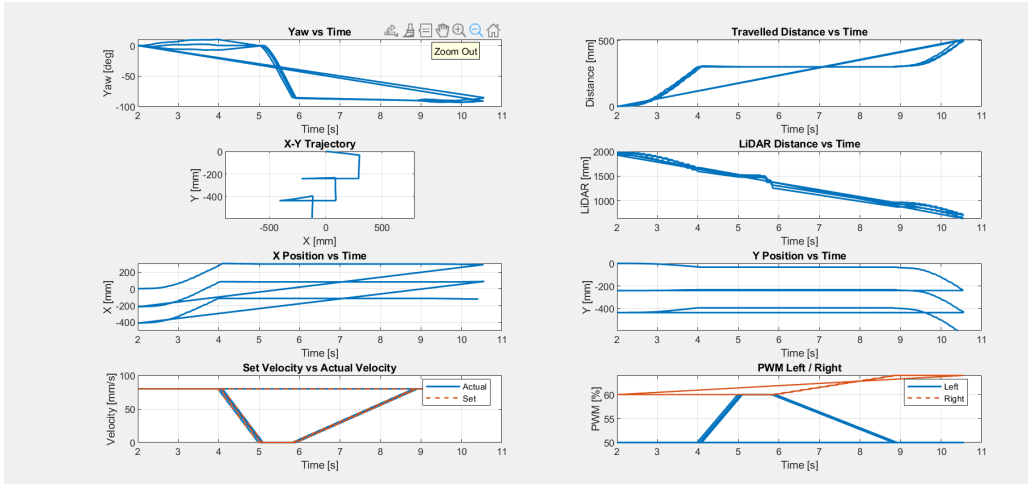


Figure 5.7: Latest Raspberry Pi to MATLAB UDP dashboard used to inspect the real half-rectangular experiment.

Figure 5.7 shows the latest Raspberry Pi to MATLAB data path. This figure is included here because it documents the implemented communication workflow, while the detailed numerical interpretation is presented in the evaluation chapter.

Figure 5.8 shows the RViz live visualisation workflow. The RViz display is used as a demonstration and validation tool for the UDP-to-ROS2 pipeline.

5.5 Sensor Fusion and Motion Sequence

The sensor fusion implementation is based on the EKF principle. The estimator uses the available inertial information for prediction and LiDAR-based distance information for correction. The state estimate contains the robot position, velocity components, and yaw angle. During straight motion, the LiDAR distance change provides a reference for correcting the estimated travelled distance. During turning, the IMU heading information is used to observe the yaw behaviour.

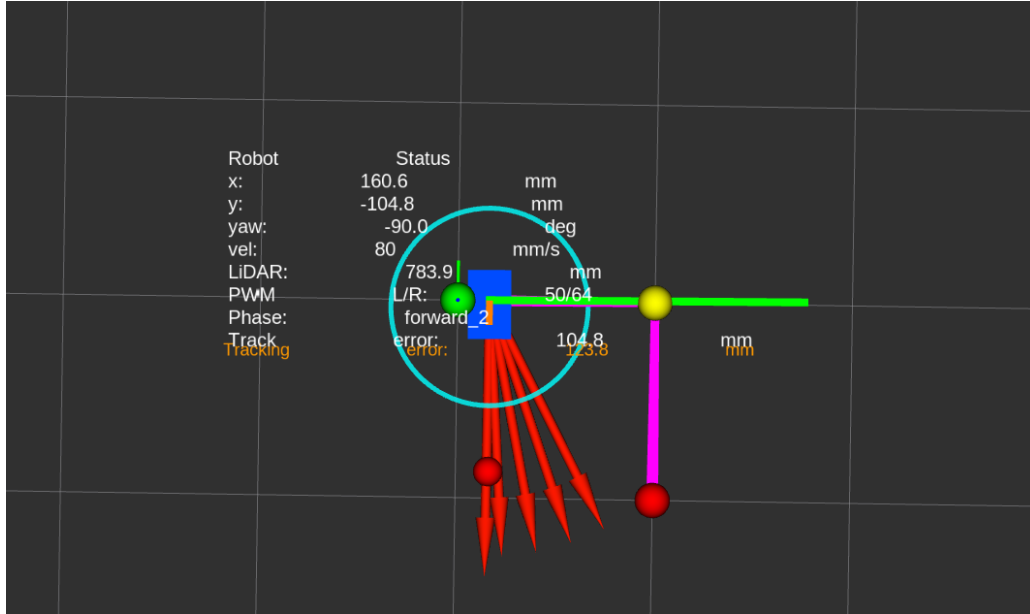


Figure 5.8: RViz live visualisation of the UDP telemetry after conversion into ROS2 messages on Ubuntu.

The latest physical motion sequence contains three phases only. In the first phase, **forward_1**, the robot moves forward with a target distance of 300 mm. In the second phase, **turn_right**, the robot turns toward a right-angle orientation of approximately -90° . In the third phase, **forward_2**, the robot moves forward again with a target distance of 200 mm. This sequence represents the first half of the intended rectangular path. A full rectangular path can be produced by repeating the same basic motion primitives, but the complete four-side route is not claimed as a measured result in this report.

The EKF prediction step is updated from the motion model, while the correction step uses available measurements to reduce drift. The general update follows the Kalman filtering structure introduced in Chapter 2. Zero-velocity handling is applied during stationary periods so that the estimated velocity does not continue to grow while the robot is stopped. In the latest implementation, the estimator is therefore used as a practical drift reduction method rather than as a full map-based localisation system.

5.6 Motor Control Implementation

The motor control module translates the motion phase into GPIO direction signals and PWM values. The system uses different PWM settings for the latest motion phases because the two motors do not behave identically. Mechanical imbalance, friction, load, and battery voltage can cause one side of the robot to move faster than the other. Therefore, the right motor uses a slightly higher PWM value during forward

motion to support straighter travel.

The latest real test uses the phase-dependent PWM values shown in Table 5.2. These values replace the older calibration description and are based on the latest half-rectangular experiment.

Table 5.2: PWM values used in the latest half-rectangular experiment.

Motion phase	Left PWM	Right PWM	Purpose
forward_1	50	60	First 300 mm forward movement
turn_right	60	60	Right-angle turning phase
forward_2	50	64	Second 200 mm forward movement

The direction-control logic is implemented using GPIO outputs to the L298N input pins. Table 5.3 shows the general control logic. Only the forward and right-turn states are used in the latest physical experiment, but the remaining states are part of the implemented motor-control module and can be used for future trajectory extensions.

Table 5.3: GPIO outputs for motor direction control.

State	IN1	IN2	IN3	IN4	ENA	ENB
FORWARD	HIGH	LOW	HIGH	LOW	PWM	PWM
BACKWARD	LOW	HIGH	LOW	HIGH	PWM	PWM
TURN LEFT	HIGH	LOW	LOW	HIGH	PWM	PWM
TURN RIGHT	LOW	HIGH	HIGH	LOW	PWM	PWM
STOP	LOW	LOW	LOW	LOW	LOW	LOW

The motor-control strategy is intentionally simple because the main objective of this work is not precision motor control with encoders, but encoder-free navigation using sensor fusion. The PWM commands generate the intended motion, while the IMU, LiDAR, and EKF provide the information required to evaluate and correct the estimated movement.

5.7 Integration and Testing Strategy

The system was integrated in stages. First, the individual hardware components were checked separately. The LiDAR was tested for valid distance readings, the IMU was checked for stable acceleration and heading information, and the motors were tested with GPIO direction and PWM commands. The voltage monitoring path was also checked to ensure that the power supply remained stable during motor operation.

After component-level checks, the sensing, estimation, and motor-control software were combined. The robot was then tested in a controlled indoor area where a visible reference surface was available for LiDAR correction. The latest full-system validation is the L-shaped half-rectangular experiment. In this experiment, the robot executes

`forward_1`, `turn_right`, and `forward_2`. The Raspberry Pi transmits telemetry over UDP, and the data are analysed using MATLAB and visualised using ROS2/RViz.

The previous idea of validating the system using ten repeated complete rectangular runs is not used as a final experimental claim in this report, because the latest real data do not contain all four sides of a rectangle. Instead, the latest implementation is validated using the available half-rectangular data and simulation comparison. The complete rectangular route is kept as a direct future extension of the same motion sequence.

5.8 Summary

This chapter presented the design and implementation of the autonomous navigation vehicle. The mechanical platform, circuit design, vehicle assembly, software workflow, EKF-based estimation, motor-control strategy, and communication pipeline were described. The original hardware design images were retained because they document the actual physical system. The motion data were updated to reflect the latest available experiment, which is an L-shaped, half-rectangular test rather than a completed full rectangular trajectory.

The next chapter evaluates the implementation using the latest Raspberry Pi to MATLAB UDP data, the Ubuntu/ROS2/RViz visualisation workflow, and the EKF versus no-EKF comparison.

Chapter 6

Evaluation

This chapter evaluates the implemented autonomous navigation system using the latest available experimental data. The evaluation is based on the real Raspberry Pi to MATLAB UDP dataset, the Ubuntu/ROS2/RViz visualisation results, and the EKF versus no-EKF comparison images. The latest physical experiment is an L-shaped, half-rectangular motion sequence consisting of `forward_1`, `turn_right`, and `forward_2`. Therefore, this chapter does not report a complete four-leg rectangular trajectory, ten repeated rectangle runs, or return-to-origin accuracy. Those metrics would only be valid after a full rectangular experiment has been recorded.

6.1 Test Environment, Data Sources, and Methodology

The physical test was carried out in an indoor environment using the Raspberry Pi robot described in the previous chapter. The robot was not equipped with wheel encoders, so the evaluation relies on the telemetry produced by the sensor-fusion and control pipeline. The main data source is the latest Raspberry Pi to MATLAB CSV log, which was transmitted through UDP and plotted in MATLAB. The dataset contains 208 samples and includes the motion phase, target segment distance, time-based distance, raw LiDAR distance, filtered LiDAR distance, LiDAR segment travel, EKF distance, used distance, set yaw, actual yaw, and left and right PWM values.

The second data source is the Ubuntu/ROS2/RViz visualisation workflow. In that workflow, UDP telemetry is received on Ubuntu, converted into ROS2 messages, and displayed in RViz. The related MATLAB plots are used to support visual inspection of the path, phase sequence, distance estimates, yaw profile, LiDAR behaviour, PWM profile, error profile, and combined summary. These plots do not replace the numerical CSV evaluation, but they provide additional validation that the live visualisation pipeline follows the same motion sequence.

The third data source is the EKF versus no-EKF comparison. This comparison is included to show the importance of sensor fusion. Since the latest physical experiment is a half-rectangular test, the comparison is used as supporting evidence for the benefit of EKF correction, while the real measured result is kept limited to the available L-shaped trajectory.

The evaluation criteria are selected according to the latest data that were actually recorded. The first criterion is segment-distance accuracy during the two forward movements. The target distance of `forward_1` is 300 mm and the target distance of `forward_2` is 200 mm. The second criterion is velocity behaviour during these two forward movements. Since the robot does not use wheel encoders, the velocity analysis is based on telemetry interpretation rather than direct wheel-speed measurement. The third criterion is yaw behaviour during and after the right turn. The set yaw for the turning phase is approximately -90° . The fourth criterion is the relationship between LiDAR travel, EKF distance, and used distance, because these signals show how the encoder-free distance estimate is formed. The fifth criterion is the correct operation of the UDP communication and visualisation workflow. The sixth criterion is the comparison between EKF-based estimation and no-EKF behaviour.

No return-to-origin error is calculated in this chapter because the latest real experiment does not contain a fourth side or a return to the starting point. No full perimeter error is calculated because the complete rectangular perimeter was not measured in the latest CSV file. This keeps the evaluation consistent with the actual dataset and avoids unsupported claims.

6.2 Latest Real Motion Result

The latest physical dataset contains three motion phases. Table 6.1 summarises the measured final values for each phase. The first forward phase reaches an EKF distance of 300.47 mm against a 300 mm target. The second forward phase reaches an EKF distance of 200.86 mm against a 200 mm target. During the turning phase, the yaw reaches approximately -85.34° , and during the second forward phase it later settles close to -89.99° .

Table 6.1: Latest real motion result from the Raspberry Pi to MATLAB UDP dataset.

Phase	Samples	Target	Final EKF	Final yaw	PWM L/R
<code>forward_1</code>	93	300 mm	300.47 mm	9.94°	50 / 60
<code>turn_right</code>	40	-90°	0.00 mm	-85.34°	60 / 60
<code>forward_2</code>	75	200 mm	200.86 mm	-89.99°	50 / 64

The result confirms that the latest real test is an L-shaped trajectory. The two forward motions are completed with EKF distance estimates very close to the com-

manded segment distances. The turn does not finish exactly at -90° at the end of the turning phase, but the yaw settles close to the desired orientation afterwards. This behaviour is reasonable for a low-cost differential-drive robot because the robot still experiences mechanical delay, surface friction, and sensor noise during the transition from turning to straight motion.

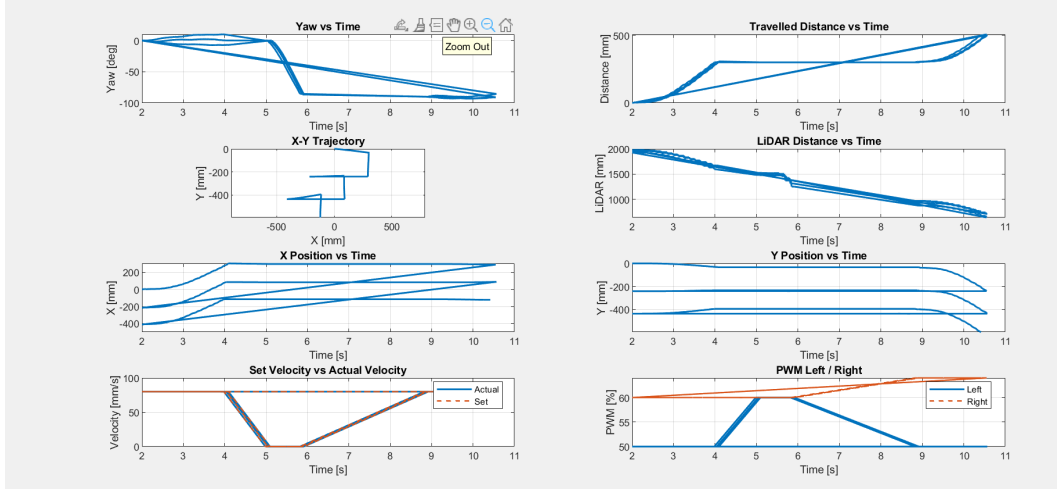


Figure 6.1: Latest Raspberry Pi to MATLAB UDP dashboard showing the real half-rectangular experiment.

Figure 6.1 provides the main overview of the latest physical experiment. It shows that the recorded data belong to the real Raspberry Pi to MATLAB path and that the experiment contains the three phases used in this evaluation.

6.3 Distance, Velocity, and Yaw Behaviour

The segment-distance evaluation compares the commanded forward distances with the final EKF distance estimates. The first forward segment has a target of 300 mm and the second forward segment has a target of 200 mm. Table 6.2 shows the resulting distance errors. The error is calculated as the final EKF distance minus the target distance.

Table 6.2: Segment-distance accuracy for the latest real forward motions.

Phase	Target	Final EKF	Error	Interpretation
forward_1	300 mm	300.47 mm	+0.47 mm	Very close to target
forward_2	200 mm	200.86 mm	+0.86 mm	Very close to target

The two forward phases show small positive errors. The first forward segment overshoots the target by 0.47 mm, and the second forward segment overshoots the target by 0.86 mm. These values indicate that the EKF-based distance estimate tracks the commanded segment distances closely in the latest experiment. However, the result

must be interpreted carefully: it validates the two measured forward segments of the half-rectangular route, not a complete rectangular course.

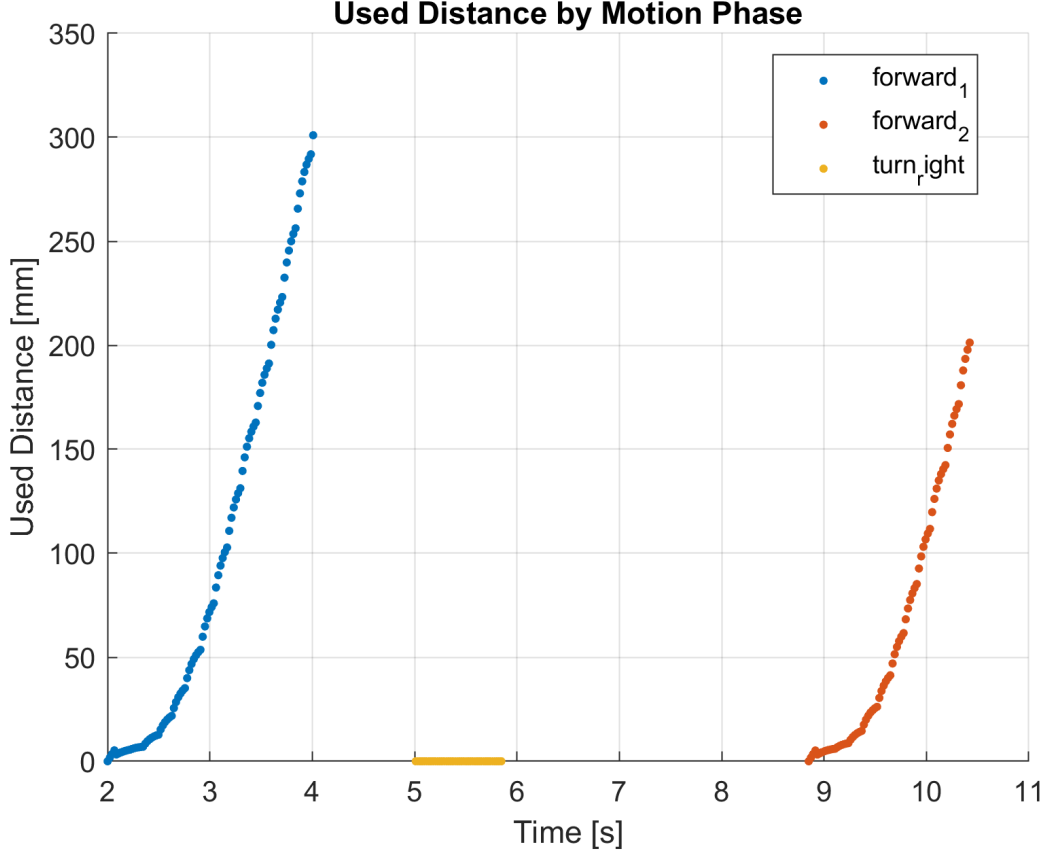


Figure 6.2: Distance-related signals from the Ubuntu/ROS2/RViz data later plotted in MATLAB.

Figure 6.2 supports the distance evaluation by showing the distance-related signals during the recorded motion. Since no wheel encoders are used, this distance plot is important for checking how the estimated motion changes over the two forward phases.

The velocity behaviour was evaluated for the two straight forward phases, **forward₁** and **forward₂**. Since the robot does not use wheel encoders, the real wheel velocity is not measured directly from wheel rotation. Instead, the velocity behaviour is interpreted from the available telemetry, including the motion phase, EKF distance, LiDAR segment travel, time-based distance, and PWM motor commands. During **forward₁**, the robot was commanded to complete the first 300 mm straight segment with left and right PWM values of 50 and 60. During **forward₂**, the robot was commanded to complete the second 200 mm straight segment with left and right PWM values of 50 and 64. The higher right PWM value in both forward phases was used to compensate for mechanical differences between the two drive sides and to support straighter motion.

Table 6.3 shows that the selected PWM commands were sufficient to complete both forward movements close to their intended segment distances. This indicates that the velocity behaviour was practically suitable for the tested L-shaped trajectory. However,

Table 6.3: Velocity-related interpretation of the two forward phases.

Phase	Target distance	Final EKF distance	PWM L/R	Velocity interpretation
forward_1	300 mm	300.47 mm	50 / 60	Forward motion complete
forward_2	200 mm	200.86 mm	50 / 64	Forward motion complete

the PWM values must not be interpreted as true speed measurements. They only describe the motor command sent to the L298N driver. The actual robot velocity still depends on battery voltage, floor friction, motor load, wheel contact, and mechanical imbalance between the left and right motors. For this reason, the velocity evaluation in this report is qualitative rather than a precise wheel-speed measurement. A more exact velocity evaluation would require either wheel encoders, an external motion-tracking system, or a calibrated independent distance-time reference.

The yaw evaluation focuses on the right-turn phase and the heading during the second forward movement. The set yaw for the turn is approximately -90° . At the end of the turning phase, the recorded yaw is approximately -85.34° , corresponding to a turn error of about 4.66 degrees relative to the desired right-angle orientation. During forward_2, the yaw later settles near -89.99° , which is very close to the target orientation.

Table 6.4: Yaw behaviour during the latest half-rectangular experiment.

Point of evaluation	Target yaw	Measured yaw	Difference
End of turn_right	-90°	-85.34°	4.66°
End of forward_2	-90°	-89.99°	0.01°

The yaw result shows that the robot does not stop exactly at the desired yaw angle immediately at the end of the turning phase, but the heading converges very close to the desired orientation afterwards. This is acceptable for the current prototype because the turn is executed by open-loop motor commands and the robot is affected by inertia, surface friction, and motor mismatch. The result also shows why continuous heading monitoring is important for the next forward segment.

Figure 6.3 shows the yaw response during the half-rectangular sequence. The plot is used to verify the right-turn behaviour and the heading during the second forward movement.

6.4 LiDAR, EKF, and Motor-Control Behaviour

The latest dataset includes raw LiDAR distance, filtered LiDAR distance, LiDAR segment travel, EKF distance, used distance, and PWM motor commands. These signals are important because the robot does not have wheel encoders. The LiDAR provides an external distance reference, while the EKF combines this information with

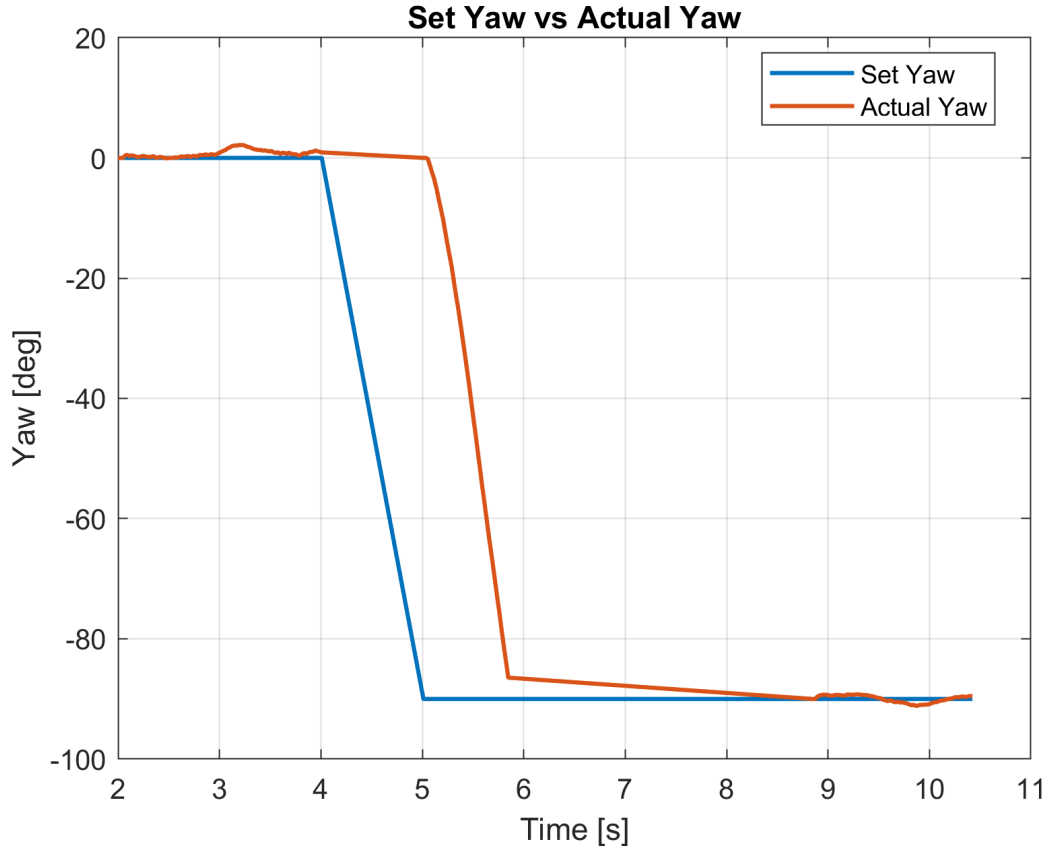


Figure 6.3: Yaw profile from the Ubuntu/ROS2/RViz data later plotted in MATLAB.

the inertial estimate. This is the main mechanism used to reduce the drift that would appear in pure dead reckoning.

During `forward_1`, the final LiDAR segment travel reaches approximately 326.43 mm while the EKF distance reaches 300.47 mm. During `forward_2`, the final LiDAR segment travel reaches approximately 204.25 mm while the EKF distance reaches 200.86 mm. The LiDAR travel therefore provides useful correction information, but the final EKF distance remains closer to the commanded target values. This shows that the EKF does not simply copy the raw LiDAR travel; it uses the available measurements as part of a filtered estimate.

Table 6.5: LiDAR segment travel and EKF distance in the latest real forward phases.

Phase	Target	Final LiDAR travel	Final EKF distance
forward_1	300 mm	326.43 mm	300.47 mm
forward_2	200 mm	204.25 mm	200.86 mm

The difference between LiDAR travel and EKF distance is expected because the LiDAR signal can contain environmental and measurement effects. The filtered state estimate is therefore more suitable for motion evaluation than a direct interpretation of raw LiDAR change alone.

Figure 6.4 shows the LiDAR-related signals. It supports the interpretation that

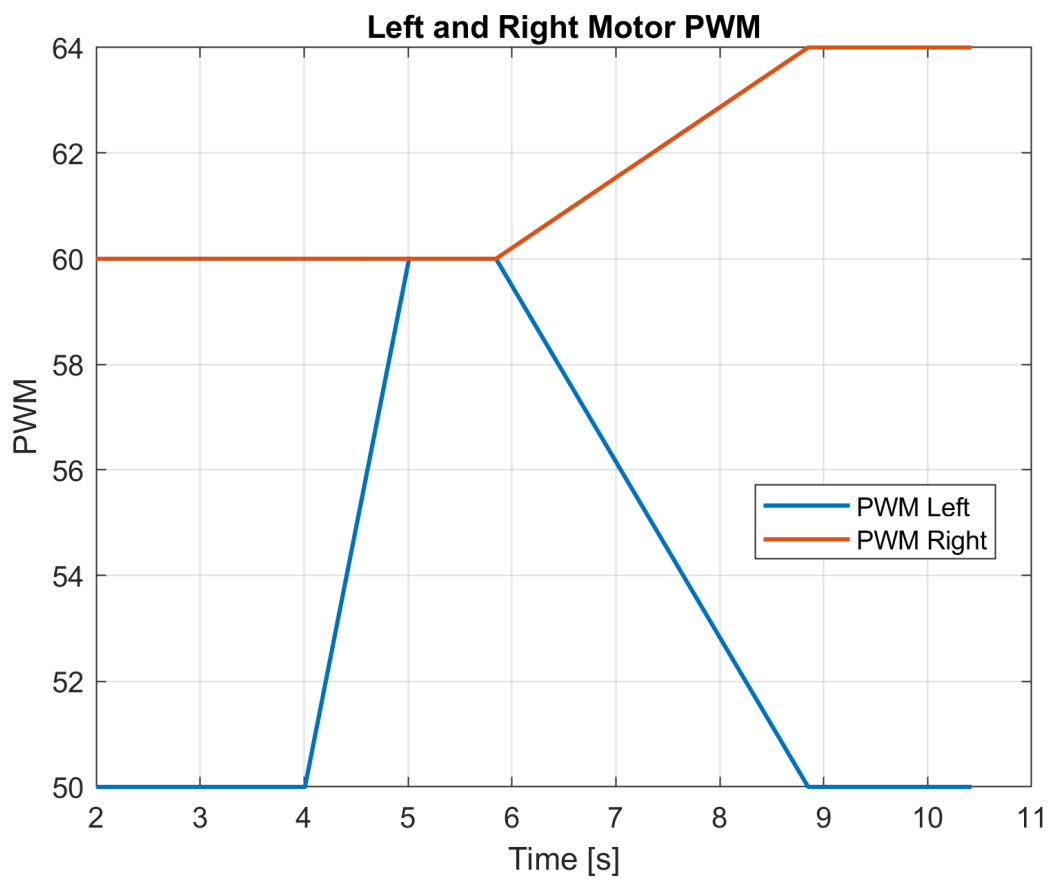


Figure 6.4: Raw and filtered LiDAR behaviour from the Ubuntu/ROS2/RViz data later plotted in MATLAB.

LiDAR provides important correction information, but still requires filtering and fusion for stable motion estimation.

The latest real test uses different PWM settings for each phase. During **forward_1**, the left and right PWM values are 50 and 60. During the right turn, both PWM values are 60. During **forward_2**, the left and right PWM values are 50 and 64. These phase-dependent values are used to drive the robot through the latest L-shaped test and to compensate partly for mechanical asymmetry between the left and right sides of the drive system.

Table 6.6: PWM settings used in the latest real experiment.

Phase	Left PWM	Right PWM	Purpose
forward_1	50	60	First 300 mm forward motion
turn_right	60	60	Right-angle turning phase
forward_2	50	64	Second 200 mm forward motion

The PWM values should not be interpreted as direct velocity measurements. They are motor commands, and the actual motion depends on battery voltage, motor load, surface friction, and mechanical differences. Therefore, the PWM profile is evaluated together with the distance and yaw estimates rather than as an independent proof of trajectory accuracy.

Figure 6.5 shows how the motor commands change during the recorded phases. This supports the explanation of the motion sequence and the open-loop motor-control behaviour.

6.5 Visualisation Pipeline and EKF Comparison

The ROS2/RViz workflow is evaluated as a live visualisation and demonstration tool. The Raspberry Pi sends telemetry using UDP, and the Ubuntu-side receiver converts the data into ROS2 messages. RViz then displays the robot state, path, markers, and related information. This confirms that the robot telemetry can be used not only for MATLAB post-processing, but also for live visual inspection.

Figures 6.6, 6.7, and 6.8 show that the telemetry can be represented in both MATLAB and RViz. This is useful for demonstration because the evaluator can see the robot state visually while the system is running. The live visualisation is not used to claim additional trajectory accuracy; it is used to support interpretation and demonstration of the implemented pipeline.

The EKF comparison is included to show why sensor fusion is necessary. The latest physical test demonstrates the real L-shaped motion, while the EKF/no-EKF comparison supports the conclusion that filtering and correction reduce drift. In a no-EKF case, the estimate depends more heavily on uncorrected motion information and

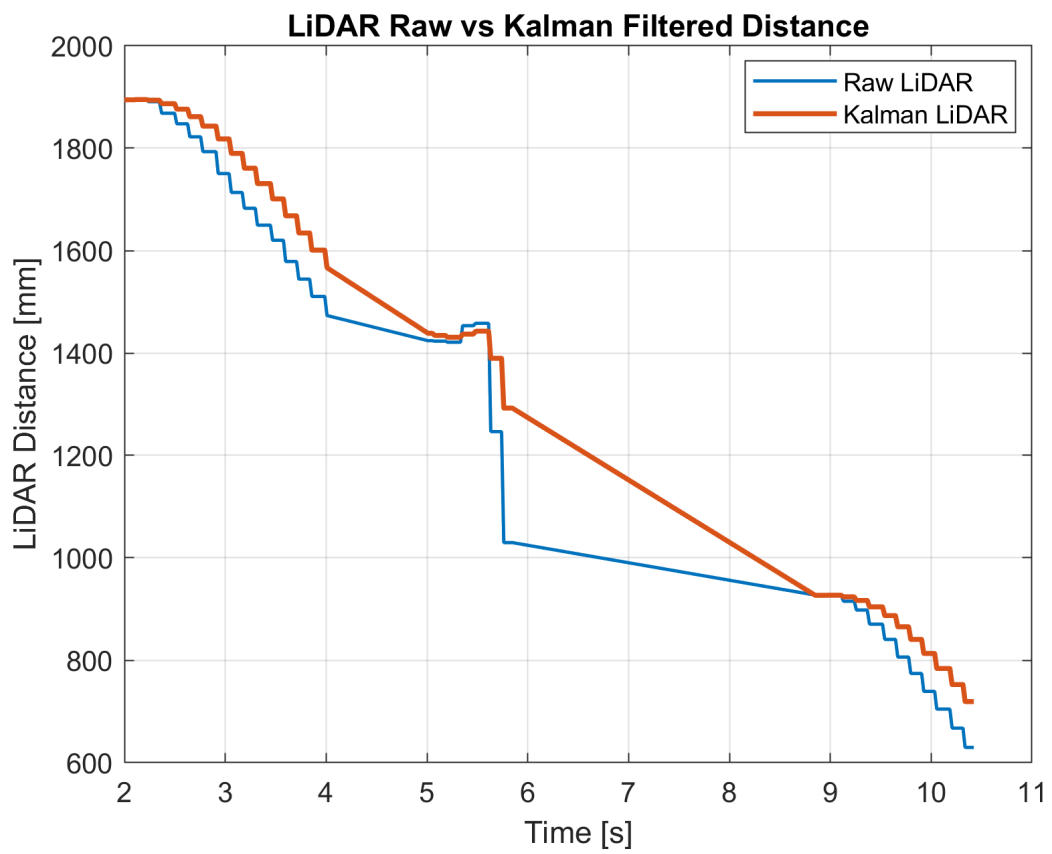


Figure 6.5: PWM command profile from the Ubuntu/ROS2/RViz data later plotted in MATLAB.

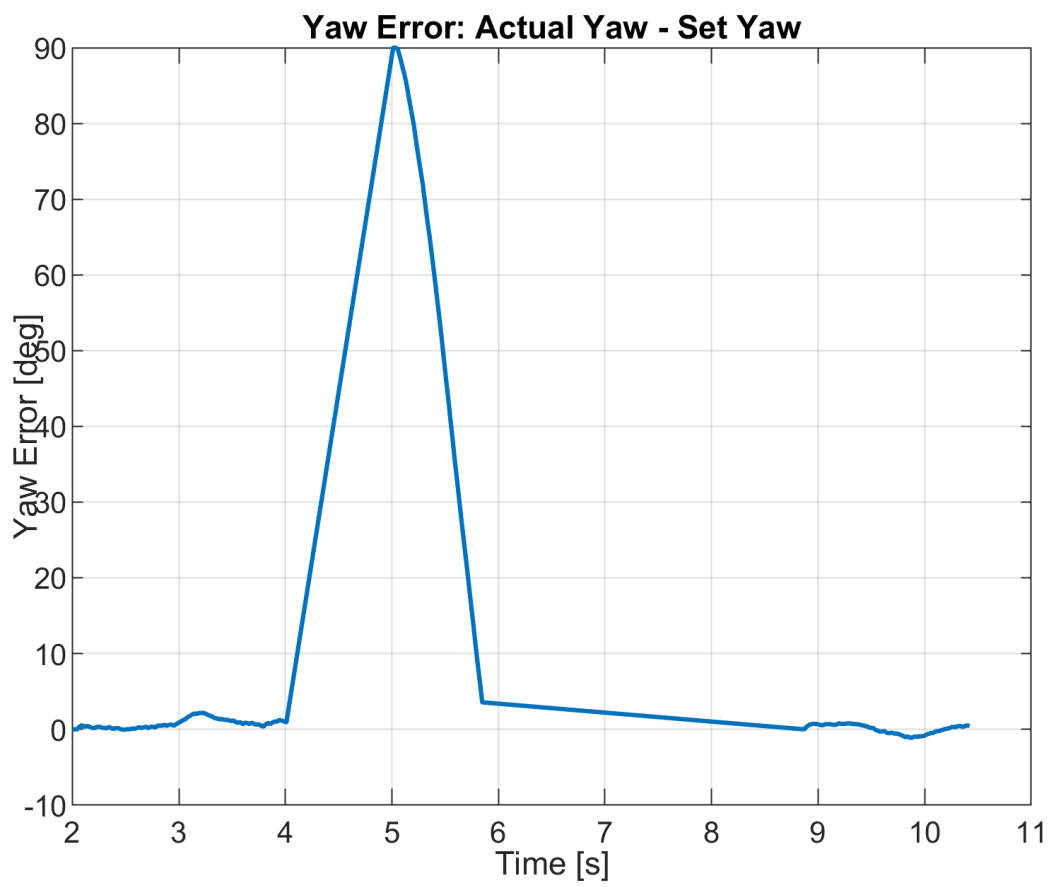


Figure 6.6: Path and phase overview from the Ubuntu/ROS2/RViz data later plotted in MATLAB.

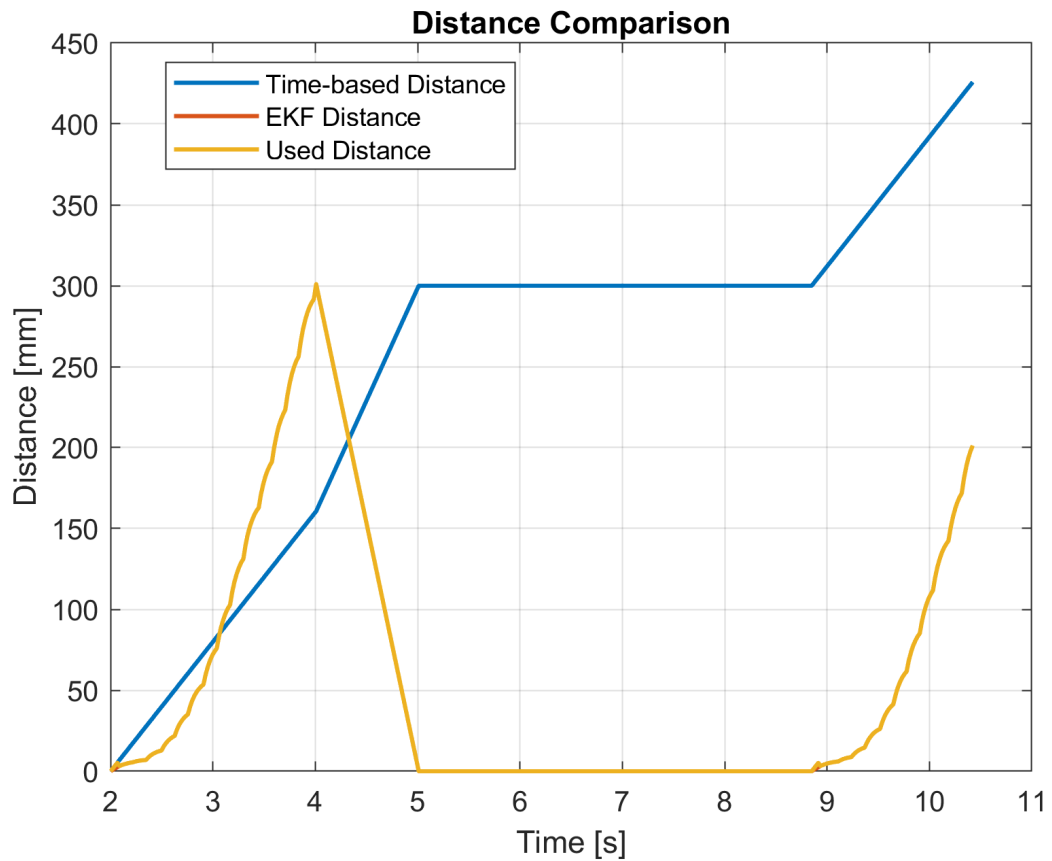


Figure 6.7: Combined summary plot from the Ubuntu/ROS2/RViz data later plotted in MATLAB.

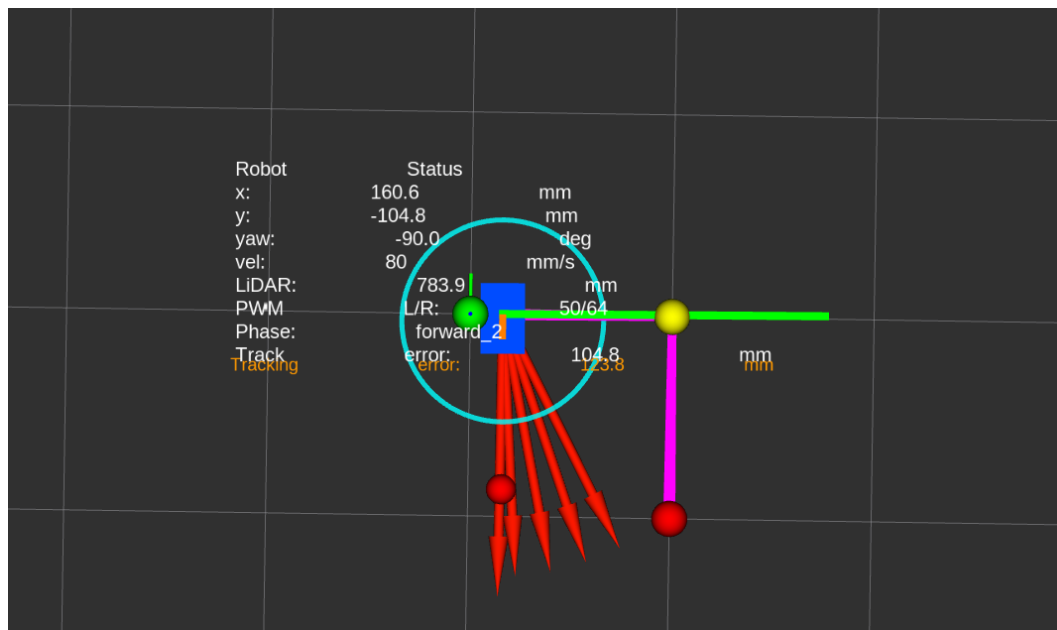


Figure 6.8: Live RViz view of the UDP telemetry after conversion into ROS2 messages.

is therefore more likely to drift away from the expected path. With EKF correction, the estimate uses available sensor measurements to reduce this drift.

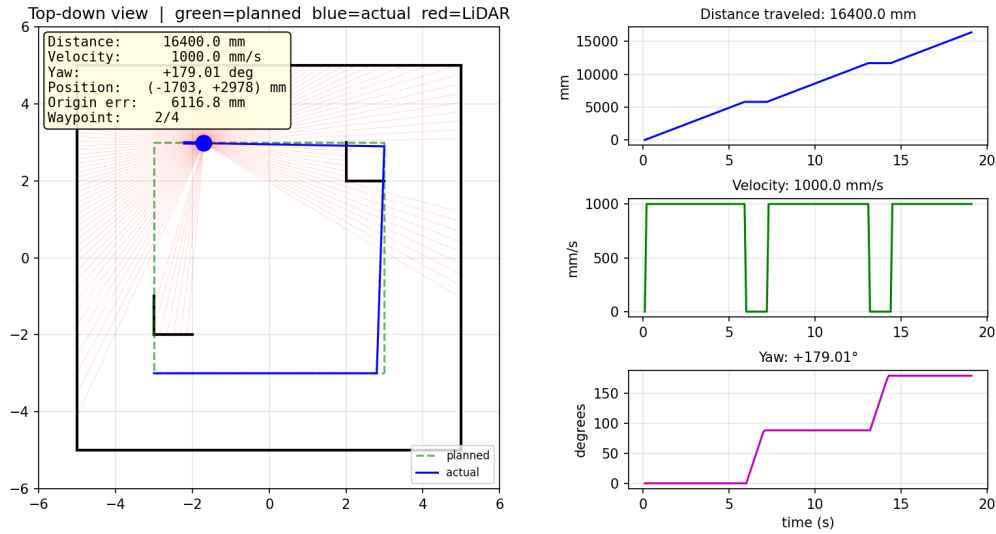


Figure 6.9: Simulation result without EKF correction, showing stronger drift in the estimated trajectory.

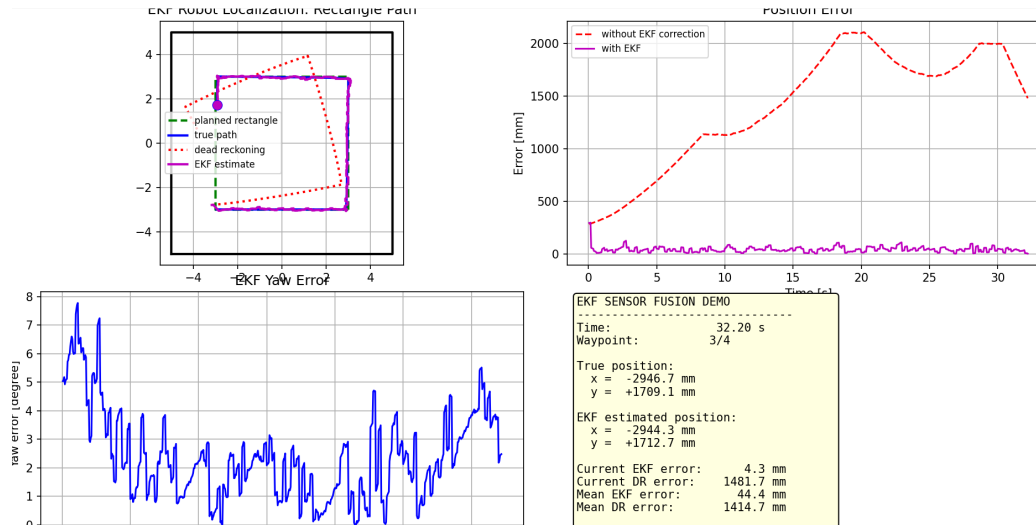


Figure 6.10: Simulation result with EKF correction, showing improved trajectory estimation compared with the no-EKF case.

The comparison figures support the interpretation that EKF correction improves the stability of the estimated trajectory. The exact quantitative evaluation in this report remains based on the latest real CSV data, while the simulation and comparison images are used to explain the role of sensor fusion.

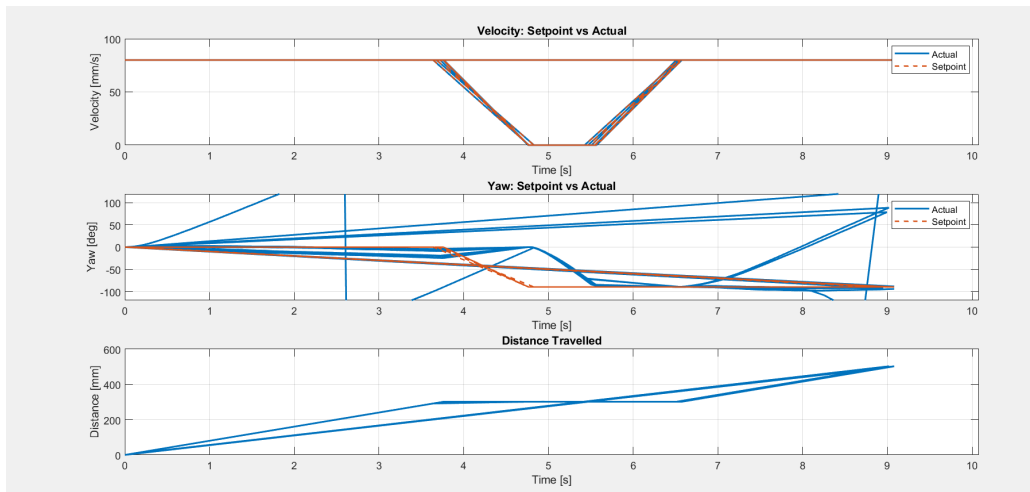


Figure 6.11: Real-life comparison image without EKF correction.

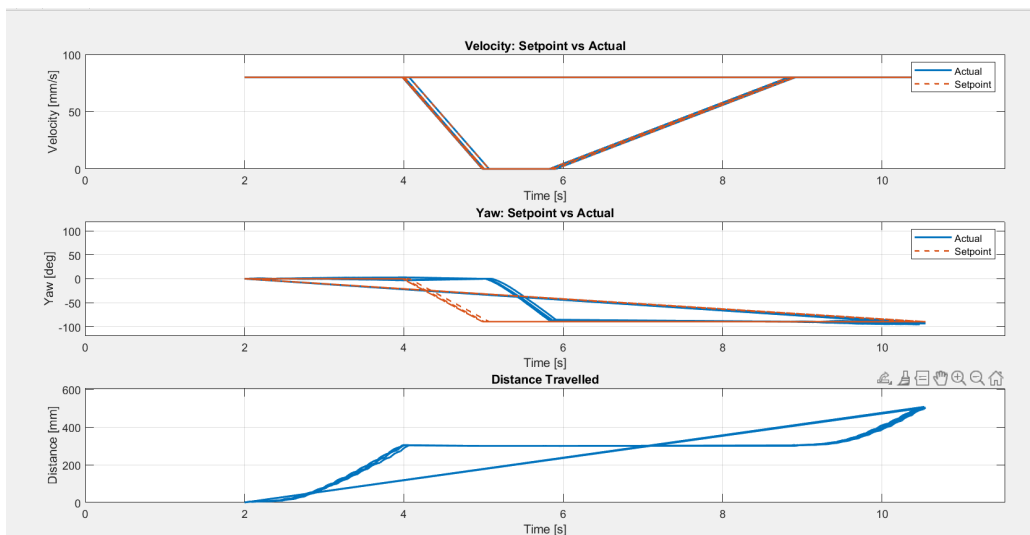


Figure 6.12: Real-life comparison image with EKF correction.

6.6 Limitations and Overall Assessment

The most important limitation is that the physical data do not yet contain the full four-leg rectangular trajectory. Therefore, return-to-origin error, full perimeter error, and ten-run repeatability are not reported as completed results. The measured L-shaped test is still valuable because it verifies the two basic motion primitives required for a rectangle: straight movement and a right turn. A complete rectangle can be produced by repeating these primitives, but the final full-rectangle claim must be supported by a new log that includes all four straight segments and all turns.

The second limitation is that the LiDAR correction depends on visible reference surfaces. If no wall or object is visible in the correct direction, the correction quality can decrease. The third limitation is that the PWM values are manually tuned, so performance may change with battery voltage, surface condition, and motor load. These limitations do not invalidate the current result, but they define what must be improved before a stronger full-rectangle evaluation can be claimed.

The latest evaluation shows that the implemented robot can execute the available half-rectangular motion sequence and produce useful telemetry for analysis. The two forward segments are estimated close to their commanded distances, and the yaw angle settles close to the intended right-angle orientation during the second forward segment. The LiDAR and EKF signals show that the system does not rely only on open-loop PWM commands, but uses sensor information to estimate the robot motion.

The evaluation also confirms that the communication pipeline works in two forms. The Raspberry Pi to MATLAB UDP path provides the main numerical data for post-processing. The Ubuntu/ROS2/RViz path provides a live visualisation of the same type of telemetry and supports demonstration of the system behaviour. Together, these workflows make the system easier to evaluate, debug, and present.

The completed four-sided rectangular trajectory is not claimed as a measured result in this chapter. The latest experiment demonstrates the first half of that trajectory and verifies the required movement primitives: forward motion and right-angle turning. Since a full rectangle can be constructed by repeating these primitives, the method is directly extendable to a complete rectangular test. However, the final proof of full-rectangle performance requires a new CSV dataset containing all four sides.

This chapter evaluated the latest available data from the autonomous navigation system. The old full-rectangle claims, ten-run tables, and return-to-origin values were removed because they are not supported by the latest real dataset. The current evaluation is based on the L-shaped half-rectangular trajectory: `forward_1`, `turn_right`, and `forward_2`. The real Raspberry Pi to MATLAB data, the Ubuntu/ROS2/RViz visualisation, and the EKF/no-EKF comparison together show that the implemented sensor-fusion and communication pipeline is functional and suitable for extension to a

complete rectangular motion test.

Chapter 7

Summary and Outlook

7.1 Summary

This project designed, implemented, and evaluated a low-cost encoder-free indoor navigation vehicle using a Raspberry Pi, LiDAR, IMU sensing, and EKF-based sensor fusion. The main constraint of the system is that the robot does not use wheel encoders. Therefore, its motion cannot be measured directly from wheel rotation and must instead be estimated from onboard sensor data. The latest physical implementation focuses on an L-shaped, half-rectangular movement sequence consisting of a 300 mm first forward segment, one right turn, and a 200 mm second forward segment.

At the beginning of the project, GPS was considered as a possible positioning source. However, GPS is not suitable for this indoor navigation task. Indoor GPS reception is weak or unavailable because satellite signals are strongly attenuated by walls, ceilings, and building structures. Even when a GPS signal is available near windows, its accuracy is usually not sufficient for a small robot moving only a few hundred millimetres. For this reason, the project moved towards an indoor sensor-fusion approach using LiDAR and IMU data instead of GPS-based localisation.

The implemented robot combines several practical subsystems. The Raspberry Pi acts as the embedded controller, reads the LiDAR and GY-87 IMU data, controls the L298N motor driver, estimates the robot state, and transmits telemetry over UDP. The received data are used in two host-side workflows. In the first workflow, the real Raspberry Pi telemetry is logged and plotted in MATLAB. In the second workflow, the UDP data are received on Ubuntu, converted into ROS 2 messages, and visualised in RViz. This creates a complete workflow from embedded sensing and control to offline analysis and live visualisation.

The latest evaluation is based only on the available evidence. The real physical data contain the phases `forward_1`, `turn_right`, and `forward_2`. The EKF distance estimate reaches approximately 300.47 mm for the 300 mm first forward segment and

approximately 200.86 mm for the 200 mm second forward segment. The right turn reaches approximately -85.34° at the end of the turning phase, and the yaw later settles close to -89.99° during the second forward movement. These results show that the implemented system can execute the tested L-shaped motion and produce useful telemetry for analysis.

The project also has several limitations. First, the robot does not use wheel encoders, so the actual wheel rotation and true wheel velocity are not directly measured. The PWM values are only motor commands, not true speed measurements. This means that the real motion can still be affected by battery voltage, surface friction, motor imbalance, and load changes. Second, the LiDAR correction depends on the availability of a suitable wall or reference surface. If the environment does not provide a stable reference, the LiDAR-based distance correction can become noisy or less reliable. Third, the IMU is a low-cost sensor module, so gyroscope bias, accelerometer noise, and magnetic disturbances can affect the yaw and motion estimate. Fourth, the latest physical experiment covers only the L-shaped, half-rectangular route. It does not yet prove complete four-leg rectangular navigation or return-to-origin accuracy.

Another limitation is that the current control approach is still relatively simple. The robot uses phase-dependent PWM values, for example different right motor values during the first and second forward motions, to compensate for mechanical behaviour. This works for the current prototype, but it is not a fully adaptive velocity-control system. In addition, UDP communication is useful for lightweight telemetry, but it does not guarantee packet delivery. For this project, UDP is acceptable because the telemetry is mainly used for monitoring, logging, and visualisation. However, for safety-critical control, a more robust communication strategy would be required.

The report was corrected to match the latest dataset. Earlier claims about ten complete rectangular runs, four-leg return-to-origin performance, and full perimeter error were removed because they are not supported by the latest real CSV data. This makes the evaluation more defensible: the quantitative statements are now linked to the actual half-rectangular test. The L-shaped route is still meaningful because it contains the two essential motion primitives required for a complete rectangular trajectory: straight-line motion and right-angle turning. Therefore, the implemented method is applicable to a full rectangular test, but the full rectangular result itself remains future work.

7.2 Outlook

The next step is to extend the same control and estimation logic from the L-shaped route to a complete rectangular trajectory. This requires recording all four straight segments and all right-angle turns in the same CSV format as the current experiment.

Once this dataset is available, the evaluation can be expanded with full path-length error, return-to-origin error, corner-to-corner accuracy, and repeatability over multiple runs. These metrics should only be added after real full-rectangle data have been collected.

A second improvement is to make the motor control more adaptive. In the current implementation, PWM values are manually tuned for the tested phases. This is sufficient for demonstrating the encoder-free concept, but the motion still depends on battery voltage, floor friction, load, and mechanical differences between the motors. A future version could add wheel encoders for comparison and closed-loop velocity control. This would make it possible to compare three cases: pure PWM motion, encoder-based odometry, and LiDAR-IMU EKF estimation. Such a comparison would make the evaluation stronger and would clearly show the advantages and limitations of the encoder-free approach.

A third improvement is to strengthen the LiDAR correction model. The current approach relies mainly on simple wall-distance changes. This works in a structured indoor environment where a suitable reference surface is visible. For more complex environments, future work could use scan matching, feature tracking, or a small occupancy-map-based correction method. This would allow the robot to correct its pose even when the wall geometry is less regular.

A fourth improvement is to improve the yaw estimation. The latest data show that the yaw reaches approximately -85.34° at the end of the turning phase and later settles close to -89.99° during the second forward segment. This behaviour is acceptable for the current prototype, but future work could improve the turn controller so that the robot stops closer to -90° immediately at the end of the turning phase. This could be achieved by using a yaw-based closed-loop turn controller instead of relying mainly on fixed PWM timing.

A fifth improvement is to revisit the GPS idea in a more suitable way. Standard GPS is not appropriate for this indoor project, but future work could compare the indoor LiDAR-IMU approach with other localisation options, such as UWB anchors, AprilTags, optical tracking, or external camera-based localisation. This would help show why LiDAR-IMU fusion is a practical choice for low-cost indoor navigation when GPS is not available.

Finally, the ROS 2/RViz pipeline can be extended beyond visualisation. The UDP bridge could publish standard odometry, path, transform, and marker topics in a more complete ROS 2 structure. MATLAB can remain the offline analysis tool, while ROS 2 can support live monitoring, demonstration, and future integration with navigation packages. This would make the project easier to compare with other mobile-robot platforms and more suitable for future robotics demonstrations.

Chapter 8

Conclusion

This project presented the design, implementation, and evaluation of a low-cost encoder-free autonomous indoor navigation vehicle using Raspberry Pi, LiDAR, IMU sensing, EKF-based estimation, UDP telemetry, MATLAB analysis, and ROS 2/RViz visualisation. The main objective was to investigate whether a small mobile robot can estimate and monitor its motion without wheel encoders by combining inertial measurements with LiDAR-based distance correction.

The latest physical experiment was an L-shaped, half-rectangular motion sequence consisting of `forward_1`, `turn_right`, and `forward_2`. The first forward segment had a target distance of 300 mm, the right-turn phase aimed for an orientation close to -90° , and the second forward segment had a target distance of 200 mm. The evaluation was corrected so that only this latest real dataset is used as physical experimental evidence. Earlier claims about ten complete rectangular runs, return-to-origin error, and a complete four-leg rectangular trajectory were removed because they are not supported by the latest CSV data.

The results show that the implemented system can execute the tested half-rectangular motion and generate useful telemetry for analysis. The EKF distance estimate reached approximately 300.47 mm during the first forward segment and approximately 200.86 mm during the second forward segment. The right-turn phase reached approximately -85.34° at the end of the turn, and the yaw later settled close to -89.99° during the second forward movement. These results indicate that the implemented LiDAR-IMU fusion approach is suitable for the tested indoor motion sequence.

The MATLAB plots and RViz visualisation confirm that the communication and monitoring pipeline works as intended. The Raspberry Pi transmits telemetry over UDP, MATLAB is used for offline plotting and analysis, and ROS 2/RViz is used for live visualisation on Ubuntu. This workflow makes the system easier to debug, evaluate, and demonstrate.

However, the project also has clear limitations. Since wheel encoders are not used, PWM commands cannot be interpreted as true wheel-speed measurements. The Li-

DAR correction depends on a visible wall or reference surface, and the IMU can still be affected by gyroscope bias, accelerometer noise, and magnetic disturbances. In addition, the latest real experiment does not yet contain a complete four-leg rectangular route. Therefore, full path-length error, return-to-origin error, and repeatability over multiple complete runs remain future evaluation tasks.

Overall, the project demonstrates a functional encoder-free indoor navigation pipeline on a low-cost robotic platform. The implemented system provides a practical foundation for extending the L-shaped half-rectangular test to a full rectangular trajectory in future work.

Bibliography

- [1] Johann Borenstein and Liqiang Feng. Measurement and correction of systematic odometry errors in mobile robots. *IEEE Transactions on Robotics and Automation*, 12(6):869–880, 1996.
- [2] Honeywell International Inc. *HMC5883L 3-Axis Digital Compass IC Datasheet*. Honeywell International Inc., 2013.
- [3] InvenSense Inc. *MPU-6000 and MPU-6050 Product Specification*. InvenSense Inc., 2013.
- [4] Rudolf E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.
- [5] John J. Leonard and Hugh F. Durrant-Whyte. Mobile robot localization by tracking geometric beacons. *IEEE Transactions on Robotics and Automation*, 7(3):376–382, 1991.
- [6] Sebastian O. H. Madgwick. An efficient orientation filter for inertial and inertial/magnetic sensor arrays. *Report x-io and University of Bristol*, 2010.
- [7] Robert Mahony, Tarek Hamel, and Jean-Michel Pflimlin. Nonlinear complementary filters on the special orthogonal group. *IEEE Transactions on Automatic Control*, 53(5):1203–1218, 2008.
- [8] Open Robotics. *ROS 2 Documentation*. Open Robotics, 2024.
- [9] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y. Ng. ROS: An open-source robot operating system. In *ICRA Workshop on Open Source Software*, 2009.
- [10] Raspberry Pi Foundation. *Raspberry Pi Documentation*. Raspberry Pi Foundation, 2023.
- [11] Raspberry Pi Foundation. *Raspberry Pi GPIO Documentation*. Raspberry Pi Foundation, 2023.

- [12] Roland Siegwart, Illah R. Nourbakhsh, and Davide Scaramuzza. *Introduction to Autonomous Mobile Robots*. MIT Press, 2 edition, 2011.
- [13] SLAMTEC. *RPLIDAR A1 Low Cost 360 Degree Laser Range Scanner Datasheet*. Shanghai Slamtec Co., Ltd., 2019.
- [14] STMicroelectronics. *L298 Dual Full-Bridge Driver Datasheet*. STMicroelectronics, 2000.
- [15] STMicroelectronics. *L298N Dual Full-Bridge Driver Datasheet*. STMicroelectronics, 2000.
- [16] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, 2005.
- [17] David H. Titterton and John L. Weston. *Strapdown Inertial Navigation Technology*. The Institution of Engineering and Technology, 2 edition, 2004.
- [18] Greg Welch and Gary Bishop. An introduction to the Kalman filter. Technical Report TR 95-041, University of North Carolina at Chapel Hill, 1995.

Appendix A

Appendix

This appendix contains supplementary material related to the implementation and evaluation of the autonomous navigation system. The main report focuses on the latest physical experiment, which is the L-shaped, half-rectangular test sequence. Therefore, the appendix also follows the same rule: it documents only the latest available project material and does not add unsupported claims about a completed four-leg rectangular trajectory.

A.1 Latest Experiment Data

The latest physical experiment contains the three phases `forward_1`, `turn_right`, and `forward_2`. The first forward phase represents the 300 mm straight segment, the right-turn phase represents the approximately -90° heading change, and the second forward phase represents the 200 mm straight segment. Together, these phases form an L-shaped, half-rectangular trajectory.

A complete four-leg rectangular test is not included in the latest dataset. For this reason, return-to-origin error, full perimeter error, and repeatability over several full rectangular runs are not included as final experimental results. These values should only be added in a future version after a new CSV file has been recorded with all four straight segments and all turns.

A.2 Project Files

The project files consist of the Raspberry Pi robot-control code, the UDP telemetry sender, the MATLAB receiver and plotting scripts, the ROS 2/RViz visualisation node, the recorded CSV data, and the report figures. The Raspberry Pi code is responsible for reading the LiDAR and IMU sensors, executing the motion sequence, applying EKF-based estimation, controlling the L298N motor driver, and transmitting telemetry over

UDP.

The MATLAB files are used for offline analysis of the received telemetry. They generate the plots used in the report to explain the distance estimates, yaw behaviour, LiDAR readings, PWM commands, and summary of the latest half-rectangular test. The ROS 2/RViz files are used for live visualisation on Ubuntu. They convert the UDP telemetry into visual robot-state information so that the motion can be inspected during demonstration.

A.3 Report Figures

The figures used in the report are stored in the project image folder. The hardware-related figures document the physical construction of the robot, including the component layout, chassis view, circuit diagram, assembled vehicle, and front view. The experiment-related figures document the latest Raspberry Pi to MATLAB UDP dashboard, the MATLAB plots generated from the recorded data, and the RViz live visualisation.

The simulation and comparison figures are used only to support the explanation of EKF behaviour. They show the difference between motion estimation with and without EKF correction. The real physical evaluation, however, is based on the latest CSV data from the L-shaped experiment.

A.4 Future Full-Rectangle Dataset

For a future full-rectangle evaluation, the robot should record a new dataset with the same telemetry structure as the current experiment. The required phases should include all four straight segments and all right-angle turns. After such a dataset is available, the evaluation can be extended with corner-to-corner error, full path-length error, return-to-origin error, and repeatability over multiple runs.

This appendix therefore supports the corrected structure of the report. It keeps the project documentation consistent with the latest available data and avoids adding results that were not measured in the current experiment.